

UAB

Universitat Autònoma de Barcelona

Faculty of Science

Implementation and Validation of an LLM-Based Planner for Modular ROS4HRI Robotic Systems

Juan David Bendek Williamson

Supervised by Raquel Ros

Institution / Lab: IIIA-CSIC / Universitat Autònoma de Barcelona

*Thesis submitted as a requirement for the degree of
MSc in Modelling for Science & Engineering*

Academic year: 2025–2026

Date: July 13, 2026

Abstract

Large language models have shown strong potential for high-level reasoning, instruction understanding, and human-robot interaction. However, their integration into robotic systems remains challenging when language-driven decisions must be grounded in the robot’s actual state, validated before execution, and adapted when the environment or the user’s goals change. This thesis investigates the implementation and validation of a modular LLM-based planning architecture for ROS4HRI-compatible robotic systems, using the NAO robot as the main experimental platform.

The proposed system separates dialogue, symbolic grounding, task planning, and deterministic execution. A dialogue layer handles user-facing interaction and semantic intent declaration; a planner layer generates structured plans over a reviewed skill registry; and an orchestrator validates and executes admissible actions while returning execution feedback to the planner. The work focuses on multi-intent instructions, feedback-driven planning, failure handling, clarification requests, and the use of symbolic scene context through a knowledge and grounding pipeline.

The expected contribution is a reusable and inspectable planning framework for grounded human-robot interaction, together with an empirical evaluation of its robustness, modularity, and limitations. The thesis will compare direct execution against planner-mediated execution and analyse how skill contracts, execution feedback, and grounded context influence task completion, replanning, and interaction quality.

Keywords: Human-Robot Interaction, ROS4HRI, Large Language Models, Task Planning, NAO Robot, Symbolic Grounding, Execution Monitoring, Replanning.

Acknowledgements

This page is reserved for the final acknowledgements.

Contents

Abstract	i
Acknowledgements	ii
List of Figures	vii
List of Tables	viii
List of Abbreviations	ix
1 Introduction	1
1.1 Planning for HRI	1
1.2 Problem Statement	2
1.3 Objectives	2
1.4 Research Questions	2
2 Background and Literature Review	4
2.1 Human-Robot Interaction and ROS	4
2.2 NAO Robot Stack	5
2.3 LLMs as High-Level Planners	5
2.4 Embodied Multimodal Models, VLMs, and VLAs	6
2.5 Symbolic State, Knowledge Bases, and Grounded Context	7
2.6 Execution Monitoring, Replanning, and Clarification	7
2.7 Literature Gap	8
3 System Requirements and Architecture	9
3.1 Functional Requirements	9
3.2 Design Principles	10

3.3	High-Level ROS Architecture	10
3.3.1	Runtime Flow	12
4	Runtime Contracts	13
4.1	Runtime Ownership Rules	13
4.2	Contract Map and Field Presentation	13
4.3	Contract 1: Chatbot Turn Output	14
4.4	Contract 2: Planner Request	16
4.5	Contract 3: Grounded Context	18
4.6	Contract 4: Shared Skill Entry	19
4.7	Grounded Context Pipeline	20
4.8	Planner LLM Internals	21
4.9	Contract 5: Planner Output	22
4.10	Contract 6: Skill Execution Result	24
4.11	Contract 7: Execution Feedback	24
4.12	Contract 8: Planner Dialogue Act	25
4.13	Contract 9: Raw Evidence	25
5	Implementation: ROS Nodes and APIs	27
5.1	Runtime Flow	27
5.2	ROS Nodes	30
5.2.1	<code>dialogue_manager</code>	30
5.2.2	<code>chatbot_llm</code>	30
5.2.3	<code>planner_llm</code>	32
5.2.4	Goal-Keyed Planning-State Helper	33
5.2.5	Skill Registry (<code>skill_common</code>)	33
5.2.6	<code>nao_orchestrator</code> and the Execution Boundary	34
5.3	Deterministic Guards and Bounded Fallbacks	35
5.4	Grounding and Knowledge	36
5.5	Fake Skills and Simulation Seams	37
5.6	Models and Runtime Configuration	38
6	Validation Methodology	40

6.1	Evaluation Aim	40
6.2	Validation Goals	40
6.3	Questionnaire-Based Runtime Checks	42
6.4	Fake-Skill Scenario Validation	42
6.5	Multi-Step Deterministic Fake-Skill Scenarios	43
6.6	Execution Profiles	44
6.7	Metrics	44
6.8	Evidence Bundle	45
7	Results	47
7.1	Current Status of the Validation Evidence	47
7.2	Runtime Stack Health	47
7.3	Action-Level Fake-Skill Results	48
7.4	Planner Request Admission	49
7.5	Speech Ingress and User-Turn Responsiveness	49
7.6	Results Matrix	49
7.7	Interpretation	50
8	Discussion	51
8.1	Architectural Findings	51
8.2	Planner Behaviour	51
8.3	Grounding and World State	52
8.4	ROS4HRI Modularity	52
8.5	Validation Implications	52
9	Limitations and Future Work	53
9.1	Current Limitations	53
9.2	Future Work	53
10	Conclusion	54
A	Full Runtime Contracts	55
A.1	Runtime Skill Inventory	55
B	Launch Commands	58

C Validation Log Template	59
Bibliography	60

List of Figures

3.1	High-level layered architecture. Dialogue interpretation and symbolic grounding provide parallel inputs to the planning layer. The chatbot layer also consumes symbolic scene context. The planner proposes structured skill plans, while the execution layer validates and dispatches skills before returning feedback for replanning or dialogue.	11
3.2	Runtime message and responsibility sequence from user input to spoken response. The colour coding follows Figure 3.1: dialogue components are blue, grounding and evidence are orange, planning is green, and execution is purple.	12
4.1	Grounded-context pipeline. Scene and KB evidence are projected by a grounded perception helper into a compact <code>grounded_context</code> for <code>chatbot_llm</code> and <code>planner_llm</code> . Skills receive planner-approved arguments through execution, while also retaining access to raw scene and KB evidence for live requery and validation before reporting outcomes.	21
5.1	Full-stack implementation of the NAO ROS4HRI bridge. Dialogue components are blue, grounding and evidence components orange, planning components green, execution components purple, and skills or adapters neutral grey. The <code>interaction_sim</code> and KB-fixture paths provide controlled validation inputs. The dashed red World Model Enricher box denotes future work rather than an implemented runtime component.	29
5.2	Internal <code>chatbot_llm</code> path. The node owns semantic interpretation and candidate planner metadata, but not plan construction, dispatch, or speech execution.	31
5.3	Internal <code>planner_llm</code> path, from admitted request through validated plan envelope and feedback-driven goal-state handling.	32
5.4	The <code>nao_orchestrator</code> admission and execution boundaries. All effects cross deterministic validation and retain goal and plan lineage.	35
5.5	Combined perception and symbolic-grounding pipeline. Compact projections serve dialogue and planning, while raw and fresh evidence remains at its owning seam.	37

List of Tables

3.1	Functional requirements for the proposed ROS4HRI planner stack.	9
4.1	Runtime contract map and ownership in the active dialogue, planning, and execution stack.	14
4.2	Fields in Contract 1: chatbot turn output.	15
4.3	Fields in Contract 2: minimal planner-facing request.	16
4.4	Fields in Contract 3: grounded context.	19
4.5	Shared skill-entry fields used by planning and execution.	20
4.6	Fields in Contract 5: planner output.	23
5.1	Planner-visible runtime skills and their current execution profiles.	34
5.2	Principal integrated launch profiles used by the implementation.	38
6.1	Validation goals for planner-mediated ROS4HRI execution.	42
6.2	Tailored runtime questionnaire used for end-to-end validation.	42
6.3	Scenario matrix for deterministic fake-skill validation.	43
7.1	Observed runtime health during the live-stack validation pass.	48
7.2	Preliminary fake-skill scenario results from the action-level runtime probe. . .	48
7.3	Planner request-admission observations from the preliminary runtime pass. . .	49
7.4	Current validation status matrix.	50
A.1	Compressed runtime skill inventory and registry values.	56
C.1	Validation log template.	59

List of Abbreviations

Abbreviation	Meaning
API	Application Programming Interface
ASR	Automatic Speech Recognition
HRI	Human-Robot Interaction
HTN	Hierarchical Task Network
JSON	JavaScript Object Notation
KB	Knowledge Base
LLM	Large Language Model
NAO	Humanoid robot used as the main experimental platform
PDDL	Planning Domain Definition Language
ROS	Robot Operating System
ROS4HRI	ROS-based Human-Robot Interaction framework
TFM	Trabajo de Fin de Máster
TTS	Text-to-Speech
VLA	Vision-Language-Action model
VLM	Vision-Language Model
WME	World Model Enricher

Chapter 1

Introduction

1.1 Planning for HRI

Successfully creating planning and execution systems for human-robot interaction (HRI) builds on a long tradition of automated planning. Classical planning languages such as the Planning Domain Definition Language (PDDL) were introduced to make planning domains, actions, preconditions, effects, initial states, and goals explicit and reusable across planners McDermott et al. 1998. This formal view of planning shaped later hierarchical task network (HTN) approaches, where high-level tasks are decomposed into executable subtasks through domain-specific methods Erol et al. 1994; Nau et al. 2003; Georgievski et al. 2014. More recent neuro-symbolic and LLM-assisted planning work has revisited the same underlying problem from a different direction: how to connect flexible natural-language reasoning with grounded, verifiable action structures Liu et al. 2023a; Valmeekam et al. 2023; Ahn et al. 2022.

Large Language Models (LLMs) have therefore become promising natural-language operators for robot systems. They can transform user utterances into structured intent payloads, infer likely task goals, and propose multi-step action decompositions. In HRI, this is valuable because users rarely speak in planner-ready commands. They ask questions, refer to visible objects, change goals, and expect the robot to interpret context. At the same time, LLMs are not naturally grounded in the robot’s live scene state, skill availability, or execution constraints. A plan that is plausible in language may still refer to an unavailable object, use an unsupported action, or ignore a failed step.

This thesis focuses on the problem of integrating LLM-based task planning into a modular ROS4HRI robotic system centred on the NAO robot stack. The target is not to build an unconstrained prompt-to-action robot. The aim is to evaluate whether an LLM planner can operate more robustly when it is constrained by symbolic state, explicit skill contracts, deterministic validation, and execution feedback. In this architecture, natural language is used to interpret the user’s goal, but execution remains mediated by ROS interfaces, an orchestrator, and skill-level validation.

1.2 Problem Statement

Current LLM-robot integrations often face at least one of the following limitations:

- they are tightly coupled to one robot or one task domain;
- they rely on prompt-only action generation without strong execution validation;
- they do not clearly separate dialogue, planning, perception, and execution;
- they struggle with multi-intent instructions that unfold over time;
- they provide weak mechanisms for clarification, replanning, and failure recovery.

The central problem addressed in this thesis is therefore:

How can an LLM-based planner be integrated into a ROS4HRI robotic system so that it can generate and revise structured plans while preserving modularity, groundedness, and deterministic execution boundaries?

1.3 Objectives

The main objective of this investigation is to design, implement, and evaluate an LLM-based planner the NAO robot stack using principles from ROS4HRI. The planner is intended to extract user intent into known labels and natural language goals, generate structured skill plans, and remain grounded in the robot’s available capabilities, symbolic scene context, and execution feedback.

1. design and implement an LLM-based planning layer for a ROS-based robot stack;
2. define explicit runtime contracts between user intent extraction, planner admission, symbolic grounding, skill planning, execution, and feedback;
3. expose and discretize robot capabilities through a reviewed skill registry instead of direct robot-specific APIs;
4. support multi-intent and multi-step user instructions through structured plan generation;
5. constrain planner outputs through deterministic validation before robot actions are dispatched;
6. implement execution feedback and feedback-driven planning so the system can handle failed steps, ambiguous targets, clarification requests, replanning, and changes in the world state;
7. evaluate the system through representative validation scenarios involving successful execution, failure, ambiguity, replanning, and user-facing responses.

1.4 Research Questions

This thesis is guided by the following research questions:

1. **RQ1:** Can a structured LLM planner enable a ROS4HRI robot to generate coherent multi-step skill plans from natural-language user requests?
2. **RQ2:** To what extent does compact symbolic grounding help the planner refer to objects, people, and scene context in a way that is consistent with the robot’s current world state?
3. **RQ3:** Does a reviewed skill registry, combined with deterministic validation, reduce invalid planner outputs such as unsupported skills, malformed arguments, or robot-specific API calls?
4. **RQ4:** How does execution feedback affect the system’s ability to recover from failed actions, request clarification, or produce a revised plan when the world state changes?
5. **RQ5:** Which architectural boundaries are required to keep LLM-based planning inspectable, modular, and safe enough for a ROS4HRI execution stack?

Chapter 2

Background and Literature Review

2.1 Human-Robot Interaction and ROS

Human-robot interaction (HRI) studies how people and robots communicate, coordinate, and share space. In social and service robotics, the interaction is not only a command interface. The robot must interpret speech, gaze, gesture, people, objects, and task context while remaining physically safe and predictable. Early surveys in HRI and socially interactive robots describe this as a multidisciplinary problem that combines robotics, artificial intelligence, dialogue, perception, and interaction design Fong et al. [2003](#); Goodrich et al. [2007](#). This motivates an architecture in which dialogue, perception, planning, and actuation are not treated as a single monolithic process.

ROS provides a practical software substrate for this separation. A ROS system is built from nodes that communicate through typed topics, services, and actions. Topics are suitable for streams such as perception or status updates, services for request-response queries, and actions for long-running behaviours that need feedback and cancellation. ROS 2 extends this model with design choices aimed at more robust distributed systems, including improved middleware support and deployment considerations for real robots Quigley et al. [2009](#); Macenski et al. [2022](#). For this thesis, these boundaries are useful because they allow a language model to remain a high-level planner rather than a direct controller of robot hardware.

ROS4HRI builds on this idea by proposing reusable conventions and interfaces for HRI components in ROS. Mohamed et al. ([2020](#)) identify the lack of a standard human representation as a barrier to reusable HRI pipelines, particularly when perception modules must represent bodies, faces, voices, groups, and social signals. Their ROS4HRI proposal is relevant here because it treats HRI perception as a set of interoperable streams rather than as robot-specific internal state. A planner should therefore receive compact task-relevant grounding, while perception and skill nodes retain access to richer evidence when they need to execute or validate an action.

This separation also matters for safety. A physical robot should not execute an action only because a language model produced plausible text. The runtime stack should validate

that a requested skill exists, that its arguments are admissible, that the plan uses allowed interfaces, and that execution feedback is handled before further actions are dispatched. In this work, `nao_orchestrator` is kept as the deterministic execution boundary: it admits planner requests, validates plan steps, dispatches skills, and publishes execution feedback. The language model proposes structured plans, but it does not bypass the ROS execution layer.

2.2 NAO Robot Stack

The physical platform used in this thesis is NAO, a programmable humanoid robot originally developed by Aldebaran Robotics. Gouaillier et al. (2008) describe NAO as an autonomous humanoid platform designed for research, education, and RoboCup-style experimentation. Its humanoid form, onboard perception pipeline, speech capabilities, and established use in academic robotics make it suitable for studying HRI behaviours such as spoken interaction, person/object grounding, gaze-oriented actions, navigation-like skills, and feedback-driven task execution. In this thesis, NAO is treated as the embodied execution platform, while the proposed architecture is designed so that the planning layer depends on reviewed skills and ROS interfaces rather than on NAO-specific APIs.

2.3 LLMs as High-Level Planners

Large language models (LLMs) have been studied as high-level planners because they encode procedural and semantic knowledge that can be elicited from natural-language instructions. Huang et al. (2022b) show that language models can decompose high-level embodied tasks into action-like steps in simulated environments, but also show that raw language-model plans may not map cleanly to admissible actions. This problem is central for robotics: a plan that is linguistically reasonable can still be impossible, unsafe, or unsupported by the current robot.

Several approaches address this limitation by grounding the language model in a restricted action set. SayCan combines LLM task knowledge with affordance scores from pretrained skills so that proposed actions are both semantically appropriate and feasible for the robot Ahn et al. 2022. Code as Policies uses language models to synthesize robot policy code over a provided API, which can express feedback loops and spatial computations, but still depends on a carefully defined set of callable primitives Liang et al. 2022. These approaches support the design choice used in this thesis: the LLM should plan over an explicit skill registry, not over arbitrary natural-language actions.

Reasoning and acting methods further show the value of coupling language output with external feedback. ReAct interleaves reasoning-oriented text with actions that query external sources or environments Yao et al. 2022. In robotics, Inner Monologue uses language feedback from perception, success detectors, and humans to improve long-horizon embodied task execution Huang et al. 2022c. Reflexion extends the same general idea to language agents

that improve across trials through verbalized feedback and memory, without updating model weights Shinn et al. 2023. These works are relevant because they treat execution feedback as part of the planning process rather than as an afterthought.

This thesis adopts a narrower and more implementable form of closed-loop planning. The planner produces a structured skill plan, `nao_orchestrator` validates and executes the plan, skills return structured results, and `planner_llm` receives execution feedback. The planner may then request clarification, produce a revised plan, or explain failure. The system does not expose private intermediate reasoning. It exposes only the final structured plan, dialogue acts, and feedback-dependent decisions. This preserves inspectability while avoiding the risks of treating free-form reasoning text as an execution contract.

Prompt-only planning remains limited. A prompt can tell an LLM which skills exist, but it cannot guarantee that the model will always select valid skills, use valid arguments, or remain grounded in the current scene. The stack therefore uses deterministic validation around the LLM. The planner output is JSON, plan steps are checked against allowed skills, recovery policies are constrained, and execution remains bound to the orchestrator. This is the main architectural distinction between using an LLM as a planner and using an LLM as a controller.

2.4 Embodied Multimodal Models, VLMs, and VLAs

Vision-language models (VLMs) and vision-language-action models (VLAs) provide an important context for this work. VLMs connect visual observations with language, while VLAs go further by mapping visual and language inputs to robot actions. BLIP-2 shows how frozen vision encoders and frozen language models can be connected through a lightweight transformer to support vision-language generation Li et al. 2023. LLaVA uses visual instruction tuning to create a multimodal assistant capable of following image-based instructions Liu et al. 2023b. These systems are not robot controllers, but they show how visual context can be converted into language-compatible representations.

Embodied multimodal models extend this direction into robotics. PaLM-E incorporates visual and continuous sensor inputs into a language model and evaluates the resulting model on embodied tasks including robotic planning and visual question answering Driess et al. 2023. RT-2 formulates robotic actions as tokens and co-fine-tunes vision-language models on web-scale data and robot trajectories, giving the model a direct path from visual observations and instructions to action outputs Brohan et al. 2023. OpenVLA makes this family of models more accessible by releasing an open-source VLA trained on a large collection of robot demonstrations Kim et al. 2024.

The present thesis does not attempt to train an end-to-end VLA policy. That would require a different experimental design, robot demonstration data, and low-level control evaluation. The contribution here is instead a modular high-level planning architecture for a ROS4HRI-based NAO stack. VLM and VLA literature is used to motivate future extensions in which richer visual or embodied state could improve the planner’s world model. In the current system, visual and KB evidence are converted into compact symbolic context before reaching the

planner. This keeps the planner portable across robots, provided that the robot exposes equivalent skills and grounding interfaces.

2.5 Symbolic State, Knowledge Bases, and Grounded Context

Robotic planning requires some representation of the world. Pure perception streams are usually too detailed and unstable for high-level planning, while pure language descriptions risk being incomplete or stale. A symbolic knowledge base (KB) provides one way to bridge this gap. KnowRob represents objects, actions, environments, and task knowledge in a form that supports robot reasoning and querying Tenorth et al. 2013; Tenorth et al. 2017. RoboBrain similarly treats robotic knowledge as a multi-source resource that can support grounding, perception, and planning Saxena et al. 2014. Other work has explored how interaction with users can help robots acquire knowledge about their environment and ground referential expressions Bastianelli et al. 2013; Grassi et al. 2021.

The current implementation follows the same general principle, but at a smaller scale. Detector-derived observations and KB rows are projected into a compact `grounded_context`. This context contains stable entity handles, entity kind, semantic class, visibility, and a bounded set of relevant relations. The planner can then refer to entities such as `cup_1` or `anonymous_person_ehfbf` without needing raw detector scores, image coordinates, or full RDF-style triples in every prompt.

This distinction is important for HRI. A user may ask “look at the person”, “find the cup”, or “what do you see?”. The robot must connect those phrases to current entities, but the planner does not need to subscribe directly to every perception stream. Skills can still require raw scene or KB evidence at execution time. This allows the planner prompt to remain compact while preserving access to richer evidence where it is needed. A future World Model Enricher can extend this by maintaining a more explicit evolving scene state, including uncertainty, source recency, and relations between people, objects, and actions. The current system prepares for that extension by keeping grounding as a separate contract rather than embedding it inside the planner logic.

2.6 Execution Monitoring, Replanning, and Clarification

Planning in a physical robot system cannot stop at plan generation. Actions may fail, targets may disappear, the user may change the goal, and the robot may need clarification before continuing. Closed-loop methods in language-agent and embodied-planning literature support this view. Inner Monologue shows that feedback from the environment and humans can improve embodied task completion Huang et al. 2022c. ReAct shows that language models can combine action with information gathering from external tools or environments Yao et al. 2022. Reflexion shows that feedback can be stored and reused as language-level experience

across trials Shinn et al. 2023.

The implementation in this thesis uses execution feedback as a runtime contract. A skill result is not only a log message. It informs `planner_llm` whether a plan step succeeded, failed, needs user input, or should trigger replanning. The orchestrator never creates a replacement plan locally. If a step fails with a recovery policy such as `replan`, `nao_orchestrator` blocks unsafe continuation, publishes feedback, and waits for a revised validated plan before dispatching further actions. This maintains a clear separation between execution authority and feedback-driven planning.

Clarification is treated as part of the same loop. If the planner cannot safely select a target, or if execution feedback reports ambiguity, the planner emits a dialogue act rather than inventing a missing fact. The dialogue manager then realizes that act as speech. This makes user interaction part of the execution loop, not a separate fallback. The same mechanism can be used when the user’s goal changes during execution, or when the current world state no longer satisfies the preconditions for the next planned action.

2.7 Literature Gap

Existing work demonstrates several important components of language-guided robotics. LLMs can decompose tasks into action-like sequences Huang et al. 2022b. Affordance-grounded methods can restrict language-model proposals to feasible robot skills Ahn et al. 2022. Multimodal and VLA systems can connect vision, language, and action at larger scale Driess et al. 2023; Brohan et al. 2023; Kim et al. 2024. Knowledge-base systems show how symbolic state can support grounding and planning Tenorth et al. 2013; Saxena et al. 2014. Execution-feedback approaches show that planning improves when the model can react to environment or human feedback Huang et al. 2022c; Yao et al. 2022.

The gap addressed in this thesis is practical and architectural. We are aware of no existing equivalent ROS4HRI-oriented implementation that combines dialogue routing, compact symbolic grounding, LLM-based high-level planning, deterministic orchestrator validation, skill-level execution, and feedback-driven replanning in a single modular NAO stack. The aim is not to replace ROS execution with a language model. It is to place the language model at the planning and feedback layer while preserving ROS boundaries for perception, skills, validation, and speech realization.

The resulting architecture is positioned between two extremes. It is more flexible than a fixed intent-to-skill mapping because it can produce multi-step plans and react to feedback. It is more controlled than an end-to-end language or VLA policy because it exposes explicit runtime contracts and keeps execution authority inside `nao_orchestrator`. This makes it suitable for a master’s thesis implementation, while leaving a clear path toward richer world modelling and multimodal grounding in future work.

Chapter 3

System Requirements and Architecture

3.1 Functional Requirements

This section defines the requirements and design principles that guide the proposed planner stack. The aim is to establish the architectural boundaries that the implementation must satisfy: natural-language interaction should remain separate from executable robot control, LLM-generated plans should be validated before dispatch, and live execution feedback should be returned to the planner when the world state or task status changes. Section 5 has detailed node implementation

ID	Requirement
FR1	The system shall accept natural-language user input through the dialogue stack.
FR2	The system shall distinguish dialogue-only turns from execution-eligible turns.
FR3	The system shall convert execution-eligible user turns into admitted planner requests before invoking the planner.
FR4	The planner shall generate structured skill plans over an explicit skill registry.
FR5	The orchestrator shall validate planner outputs before any robot skill is dispatched.
FR6	The system shall publish execution feedback after relevant plan and step/skill events.
FR7	The planner shall support asking for clarification, replanning and plan failure when execution feedback indicates that the current plan cannot continue safely.
FR8	The system shall expose traces, logs, or runtime messages suitable for validation of the stack.

Table 3.1: Functional requirements for the proposed ROS4HRI planner stack.

3.2 Design Principles

1. **Dialogue must remain distinct from execution.** Dialogue-based turns such as greetings, conversation and queries about the robot’s world state should be remain as dialogue and must not be routed to execution unless the user explicitly requests an action.
2. **Planner doesn’t execute, it proposes structured skill plans:** `planner_llm` selects skills from the allowed registry and produces a structured plan based on the grounded scene context. It does not execute skills, speak directly, or call robot-specific APIs.
3. **The orchestrator remains the execution gate:** all robot execution is validated and dispatched downstream by `nao_orchestrator`. It admits planner requests, validates plans, dispatches skills, and publishes execution feedback.
4. **Skills own live evidence.** Perception and execution skills should query the Knowledge Base (KB), or robot state when needed. Final execution evidence remains skill-local.

3.3 High-Level ROS Architecture

The proposed architecture separates the HRI stack into interpretation, planning, grounding, execution, and feedback responsibilities, enabling a clear distinction between conversational reasoning, task-level decision making, deterministic execution, and environment-aware adaptation.

- `dialogue_manager` acts as the central dialogue ownership component, maintaining conversational state, coordinating turn-taking, managing interaction flow, and serving as the integration point between user-facing dialogue and downstream planning services.
- `chatbot_llm` is responsible for robot utterance generation, intent declaration and goal extraction during planner-relevant turns. It transforms natural language interactions into structured representations that depict the nature of the interaction. **Dialogue** and **knowledge query** turns remain conversational, **execution** turns trigger a planner request.
- `planner_llm` performs proactive task planning over the available skill registry, normalized user intents, dialogue context, and current robot world state. The planner reasons over high-level objectives, decomposes user requests into executable actions, and generates candidate plans that align with both robot capabilities and current world state constraints.
- `nao_orchestrator` provides deterministic plan validation and skill execution management. It verifies planner outputs against available skills and execution constraints, coordinates skill invocation, monitors execution progress, and emits structured execution feedback that can be consumed by both the dialogue and planning layers.
- `grounded_context` represents a detector-to-KB JSON object by linking perception outputs to semantically meaningful entities within the robot knowledge base. This grounding layer provides a shared representation of objects, locations, and contextual

information that is consumed across the chatbot, planner and execution components.

- **skill_servers** provide the execution interface that call lower-level ROS components and robot capabilities. These servers encapsulate perception, navigation, manipulation, and interaction robot actions. Skill servers allow the planner to invoke complex robot actions with structured plans. They consume raw knowledge-base updates originating from perception pipelines and execution feedback, ensuring that newly observed entities, spatial data, state changes, and environmental information for subsequent task execution.

The interaction between these packages establishes a layered architecture in which conversational understanding produces structured intents, planning generates task-level strategies, grounding enriches decisions with environmental knowledge, orchestration ensures safe and deterministic execution, and feedback mechanisms continuously update the system state for subsequent reasoning cycles.

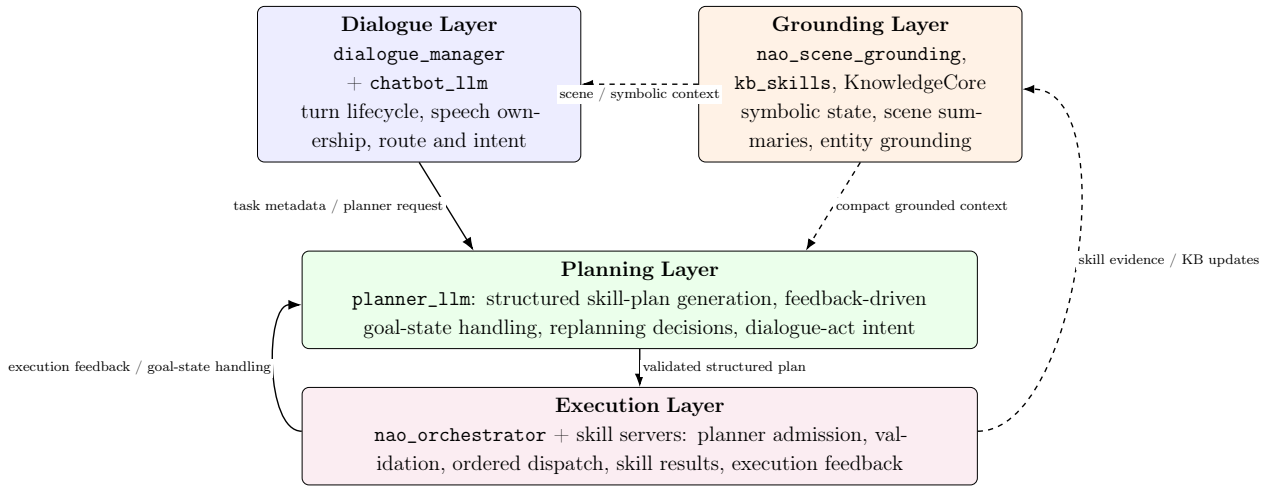


Figure 3.1: High-level layered architecture. Dialogue interpretation and symbolic grounding provide parallel inputs to the planning layer. The chatbot layer also consumes symbolic scene context. The planner proposes structured skill plans, while the execution layer validates and dispatches skills before returning feedback for replanning or dialogue.

3.3.1 Runtime Flow

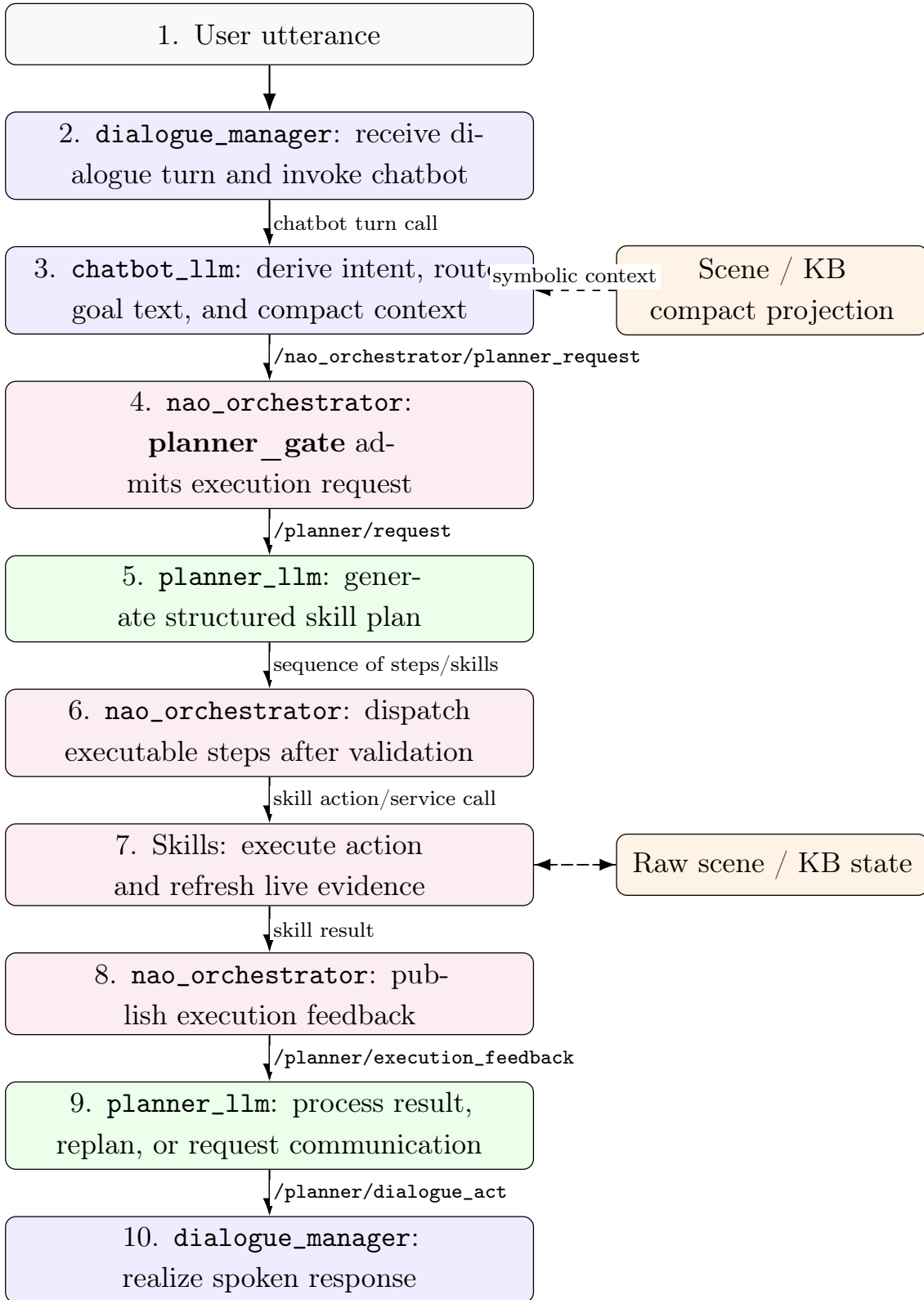


Figure 3.2: Runtime message and responsibility sequence from user input to spoken response. The colour coding follows Figure 3.1: dialogue components are blue, grounding and evidence are orange, planning is green, and execution is purple.

Chapter 4

Runtime Contracts

The architecture relies on explicit runtime contracts between dialogue interpretation, planner admission, grounding, plan generation, execution, feedback, and speech realization. These contracts are used to keep the LLM-facing parts of the system inspectable while preserving deterministic control over robot execution. Each contract defines what is exchanged, which node produces it, which node consumes it, and where validation or ownership should occur. This is important because the planner does not control the robot directly: `chatbot_llm` interprets the user turn, `nao_orchestrator` admits and validates execution requests, `planner_llm` proposes structured skill plans, skills execute with live evidence, and `dialogue_manager` realizes user-facing speech. The table below gives the contract map used throughout the rest of this section. Later subsections describe the main payloads in more detail.

4.1 Runtime Ownership Rules

- `dialogue_manager` owns the dialogue turn lifecycle and user-facing speech realization.
- `chatbot_llm` owns user-facing semantic interpretation: intent derivation, route selection (dialogue or execution) and goal text for planner.
- `nao_orchestrator` owns deterministic planner admission and request normalisation. It also owns executable-step validation, dispatch, step status, and feedback publication.
- `planner_llm` owns structured skill-plan generation, feedback-driven goal-state handling, replanning decisions, and dialogue-act intent.
- Skills own live interaction with the robot, scene, and KB evidence required to complete their operation truthfully.

4.2 Contract Map and Field Presentation

Table 4.1 lists the active runtime contracts.

Table 4.1: Runtime contract map and ownership in the active dialogue, planning, and execution stack.

ID	Contract	Interface / payload	Ownership
1	Chatbot turn output	Internal chatbot result before ROS planner publication.	<code>dialogue_manager</code> invokes the chatbot path; <code>chatbot_llm</code> produces the semantic result.
2	Planner request and admission	Candidate topic <code>/nao_orchestrator/planner_request</code> . Admitted topic <code>/planner/request</code> . Planner-facing payload is carried in <code>Intent.data</code> .	<code>chatbot_llm</code> proposes the execution request; <code>nao_orchestrator</code> admits and normalizes it; <code>planner_llm</code> consumes the admitted request.
3	Compact grounding	<code>grounded_context</code> inside Contract 2.	Scene and KB projection provide compact symbolic context for <code>chatbot_llm</code> and <code>planner_llm</code> .
4	Shared skill entry	Registry projection consumed by the planner and orchestrator; its adapter mapping identifies the receiving skill server.	<code>skill_common</code> defines the shared capability shape; <code>planner_llm</code> uses it for prompt construction and <code>nao_orchestrator</code> uses it for plan validation and dispatch.
5	Planner output	Topic <code>/intents</code> . Structured plan payload in <code>Intent.data.plan</code> .	<code>planner_llm</code> produces the plan; <code>nao_orchestrator</code> validates and dispatches executable steps.
6	Skill execution result	ROS action result or service response with a typed skill-result payload.	<code>nao_orchestrator</code> dispatches the call; the skill server owns effects and fresh evidence; verified KB effects cross <code>kb_skills</code> .
7	Execution feedback	Topic <code>/planner/execution_feedback</code> . Structured JSON feedback payload.	<code>nao_orchestrator</code> publishes feedback; a goal-keyed planning-state helper inside <code>planner_llm</code> uses it for replanning, clarification, or failure handling.
8	Planner dialogue act	Topic <code>/planner/dialogue_act</code> . Structured communication request.	<code>planner_llm</code> emits dialogue-act intent; <code>nao_orchestrator</code> relays it; <code>dialogue_manager</code> realizes the final speech.
9	Raw evidence	Topics and sources include <code>/scene/summary</code> , <code>/kb/query</code> , knowledge snapshots, skill results, and action evidence.	Scene, KB, and skill nodes provide evidence for compact projection, skill-local checks, and later feedback.

The table above lists only the contract and its meaning. Compact planner-facing fields are kept in the tables, while additional runtime metadata is described in prose when it is relevant for orchestration, feedback-driven planning, tracing, or debugging.

4.3 Contract 1: Chatbot Turn Output

The intent in this contract is the user’s goal as extracted by the chatbot. It is not a planner output.

```

1 {
2   "verbal_ack": "Okay, I will do that.",
3   "route": "execution",
4   "confidence": 0.82,
5   "user_intent": {
6     "type": "look_at",
7     "goal_text": "look at the person",
8     "scene_targets": ["person"],

```



```

9      "request_kind": "new_goal"
10    }
11  }

```

Table 4.2: Fields in Contract 1: chatbot turn output.

Field	Meaning
<code>verbal_ack</code>	Optional immediate acknowledgement generated by the chatbot layer. This may be spoken or logged before execution begins, depending on the dialogue policy.
<code>route</code>	Turn classification produced by <code>chatbot_llm</code> . Supported values include <code>dialogue</code> , <code>knowledge_query</code> , and <code>execution</code> . Only <code>execution</code> turns enter the planner path; <code>dialogue</code> and <code>knowledge_query</code> remain non-execution turns but are still useful for tracing.
<code>confidence</code>	Confidence assigned by the chatbot or intent-extraction stage. It is useful for debugging and possible admission policy, but it is not a planner priority score.
<code>user_intent.type</code>	Normalized user-intent label extracted by <code>chatbot_llm</code> , such as <code>look_at</code> , <code>find_object</code> , <code>scan</code> , or <code>navigate_to</code> .
<code>user_intent.goal_text</code>	Natural-language objective passed toward the planner path as the task to satisfy. This field preserves the user's goal in a compact form.
<code>user_intent.scene_targets</code>	Optional target hints extracted from the utterance, such as <code>person</code> , <code>cup</code> , or <code>table</code> . These are hints, not validated grounded entities.
<code>user_intent.request_kind</code>	Proposed goal transition class, such as <code>new_goal</code> , <code>update_goal</code> , <code>answer_to_clarification</code> , or <code>cancel_goal</code> . The planner gate inside <code>nao_orchestrator</code> should normalize and admit this field before publishing <code>/planner/request</code> .

4.4 Contract 2: Planner Request

The planner request is the task description generated by the orchestrator from the chatbot output and consumed by the planner

```

1 {
2   "goal_id": "goal_turn_123",
3   "request_kind": "new_goal",
4   "goal_text": "look at the person and report completion",
5   "normalized_intents": ["look_at"],
6   "grounded_context": {
7     "entities": [
8       {
9         "id": "anonymous_person_ehfbf",
10        "label": null,
11        "kind": "person",
12        "class": "Human",
13        "visible": true,
14        "relations": []
15      }
16    ]
17  }
18 }
```

“latex

Table 4.3: Fields in Contract 2: minimal planner-facing request.

Field	Meaning
<code>goal_id</code>	Identifier for the active objective. It groups replans and feedback for the same task.
<code>request_kind</code>	States whether this is a new goal, an update to the current goal, a cancellation, or another supported transition. <code>chatbot_llm</code> may propose it, but <code>nao_orchestrator</code> normalizes and admits it before publishing <code>/planner/request</code> .
<code>goal_text</code>	Objective extracted from the user utterance. This is the central planning input for <code>planner_llm</code> .
<code>normalized_intents</code>	Intent labels extracted by <code>chatbot_llm</code> . They help <code>planner_llm</code> choose suitable skills from the available skill registry.
<code>grounded_context</code>	Compact scene graph used by the planner as the current symbolic state of the world. It contains validated entities and relations rather than raw detector output.

““

Implementation metadata can still exist on the ROS envelope or runtime trace. Examples include request identifiers, dialogue turn identifiers, planner mode, parent goal ID, or super-

seded goal ID. These fields are useful for tracing, debugging, and goal management, but they are not the core task description consumed by `planner_11m`.

4.5 Contract 3: Grounded Context

The planner receives a compact, JSON-native view of relevant world state. This is best described as a compact scene graph derived from the KB, continually updated by the perception pipeline (human/person detection and object perception). Raw KB triples, detector metadata, and tracker-derived details remain available through scene/KB evidence paths to skills, but they are not duplicated into every LLM prompt.

```
1 {
2   "grounded_context": {
3     "entities": [
4       {
5         "id": "cup_jrjic",
6         "label": "cup",
7         "kind": "object",
8         "class": "Cup",
9         "visible": true,
10        "relations": [
11          {"predicate": "dbp:color", "object": "blue"},
12          {"predicate": "oro:isOn", "object": "table_1"}
13        ]
14      },
15      {
16        "id": "anonymous_person_ehfbf",
17        "label": null,
18        "kind": "person",
19        "class": "Human",
20        "visible": true,
21        "relations": []
22      }
23    ]
24  }
25 }
```

Table 4.4: Fields in Contract 3: grounded context.

Field	Meaning
<code>id</code>	Stable entity handle used by planner steps and skill arguments. This is the preferred reference when a skill needs to target a known person, object, or location.
<code>label</code>	Human-readable name or class hint. It may be empty for anonymous people or entities that should not be assigned a personal name.
<code>kind</code>	Entity category, such as <code>object</code> , <code>person</code> , or another supported runtime category. This prevents people from being treated as generic objects.
<code>class</code>	Compact semantic class used by the planner for reasoning, such as <code>Cup</code> , <code>Human</code> , or <code>Table</code> .
<code>visible</code>	Indicates whether the entity is currently grounded in the latest accepted scene or KB state. It should not be treated as a guarantee that the entity will still be available at execution time.
<code>relations</code>	Bounded list of relevant semantic relations, such as colour, containment, support, proximity, or ownership-style facts. These relations add task-relevant context without exposing the full KB.

The compact view should omit detector score, image coordinates, backend, source, and recency by default. Those values are execution evidence, not planning semantics, unless a future skill explicitly needs them in its own action arguments.

4.6 Contract 4: Shared Skill Entry

The planner and orchestrator use one shared entry shape to describe a skill before it is placed in a prompt, admitted for execution, or mapped to an action server. This entry is a capability contract, not a claim that every listed adapter is available in every launch profile. It separates the information needed to select a skill from the information needed to verify its result.

```

1 {
2   "object_id": "navigate_to",
3   "kind": "skill",
4   "category": "navigation",
5   "aliases": ["go_to", "move_to_location"],
6   "params": ["target", "location"],
7   "required_params": ["target"],
8   "preconditions": ["navigation is enabled",
9                     "target is known or resolvable"],
10  "expected_effects": ["robot arrives at the target location"],
11  "observable_success": ["status"],
12  "failure_modes": ["path_blocked", "unknown_location",
13                  "safety_disabled"],
14  "planner_guidance": ["use for a semantic destination"],

```

```

15 "result_schema": {"required": ["skill", "status", "summary_text"]},
16 "safety_flags": ["navigation"],
17 "implementation_status": "fake",
18 "robot_adapter_mapping": "fake_skills.navigate_to",
19 "supports_failure_injection": true,
20 "metadata": {"runtime_callable": true}
21 }

```

Table 4.5: Shared skill-entry fields used by planning and execution.

Field	Runtime role
object_id, kind, category, aliases	Canonical planner name, semantic group, and accepted compatibility names.
params, required_params	Typed planner-facing arguments and the subset required before admission.
preconditions	Grounding or state assumptions that must be satisfied or explicitly owned by a scenario. They do not prove a future effect.
expected_effects	Intended state changes or evidence products used to select and order skills.
observable_success	Result or trace evidence that can support completion after execution.
failure_modes	Typed outcomes used by the orchestrator and the goal-state helper to stop, clarify, or request replanning.
planner_guidance	Bounded selection and ordering guidance inserted into the planner prompt.
result_schema	Required and optional result fields, including the summary and evidence payload.
safety_flags	Safety-relevant labels used during admission and dispatch.
metadata.runtime_callable	Whether the registry permits planner-visible runtime use.
implementation_status	Current implementation class, such as first-party, fake, or hybrid.
robot_adapter_mapping	Adapter or skill server that receives the dispatched call.
supports_failure_injection	Whether controlled failure outcomes are available for validation. Launch-profile checks still establish whether the selected endpoint is active.

The same shape is projected to the planner prompt, deterministic orchestrator checks, fake-skill policies, and adapter configuration. Preconditions constrain admission, while expected effects remain hypotheses until a skill returns observable evidence. This distinction is used by `report_result`: a final summary must be derived from result payloads or trace evidence rather than copied from a planner-generated claim. The complete runtime inventory is provided in Appendix A.1.

4.7 Grounded Context Pipeline

JSON object passed (along the system prompt and skills registry) to the chatbot and planner. Acts as LLM facing grounded context representing current entities in the KB.

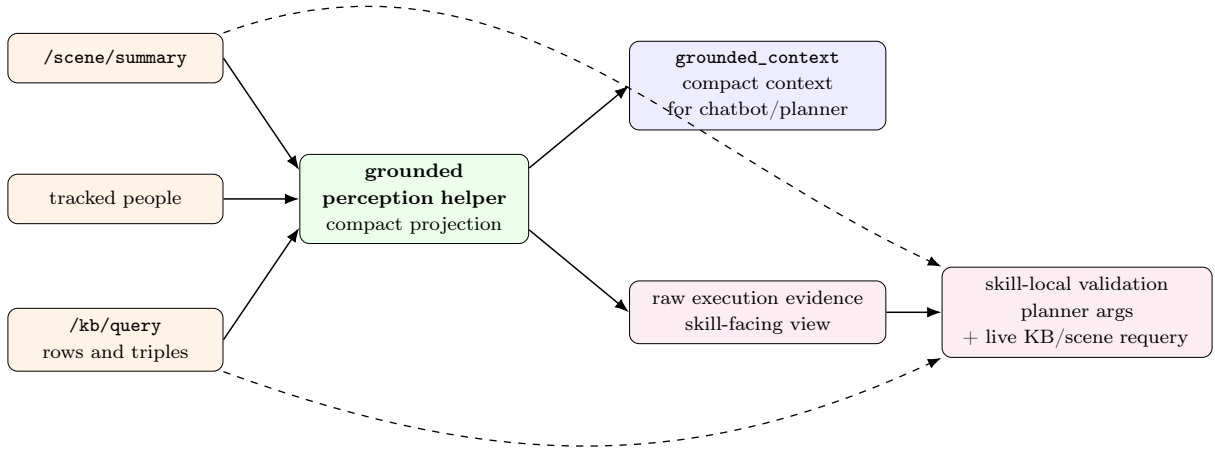


Figure 4.1: Grounded-context pipeline. Scene and KB evidence are projected by a grounded perception helper into a compact `grounded_context` for `chatbot_llm` and `planner_llm`. Skills receive planner-approved arguments through execution, while also retaining access to raw scene and KB evidence for live requery and validation before reporting outcomes.

Derivation rules:

1. Use text-derived entities only when unable to confirm entities with KB.
2. Keep anonymous people anonymous; do not invent personal names.
3. Keep IDs and labels separate.
4. Bound relation count per entity to keep prompt size manageable.

4.8 Planner LLM Internals

After the planner request is admitted, `planner_llm` builds a prompt from the compact request, grounded context, the shared skill-entry projection, allowed step types, and the output contract. The prompt carries the goal and request kind, normalized intent hints, relevant grounded entities, permitted skill names, argument requirements, and recovery vocabulary. A bounded feedback payload is added when the current goal is responding to an execution event. Rule-based shortcuts may produce deterministic plans for simple motions, but the normal path remains model-generated and contract-checked.

The provider response is handled as an untrusted candidate. The planner extracts the first admissible JSON object, normalizes plan and step metadata, validates skill names, arguments, ordering, and failure policies, then performs one bounded validation retry when the prompt pack enables it. The retry receives the validation error and a bounded portion of the previous output. A second invalid result becomes a typed planner failure or clarification outcome. No raw model text is sent to `/intents`, and no invalid plan is passed to the orchestrator.

The planner prompt should not contain raw detector coordinates, full RDF dumps, or stale perceptual reports. Those belong to skills and execution evidence.

4.9 Contract 5: Planner Output

The planner output is a plan: a sequence of skill steps to achieve the user goal. It should focus on executable content and minimal lineage. Communication policy and retry metadata are still valid runtime fields when used by the stack, but this controlled version foregrounds the ordered **steps** list because that is the essential planner output.

```
1 {
2   "plan": {
3     "goal_id": "goal_turn_123",
4     "plan_id": "plan_123",
5     "plan_version": 1,
6     "status": "planned",
7     "steps": [
8       {
9         "id": "step_1",
10        "type": "skill",
11        "name": "look_at",
12        "args": {"target_frame": "anonymous_person_ehfbf"},
13        "requires": [],
14        "on_failure": "replan"
15      }
16    ]
17  }
18 }
```


Table 4.6: Fields in Contract 5: planner output.

Field	Meaning
<code>goal_id</code>	User objective that this plan attempts to satisfy. It links the planner output to the active goal.
<code>plan_id</code>	Identifier for this concrete plan proposal. It is useful when the runtime keeps plan history or compares revised plans for the same goal.
<code>plan_version</code>	Revision number for replans of the same goal. This helps distinguish the initial plan from later feedback-driven planning outputs.
<code>status</code>	Planning outcome for this plan, such as <code>planned</code> , <code>needs_clarification</code> , or <code>failed_to_plan</code> . This describes whether <code>planner_llm</code> produced an executable plan, not whether the robot has already executed it.
<code>steps</code>	Ordered list of skill calls produced by the planner. This is the essential planner output consumed by <code>nao_orchestrator</code> .
<code>steps[].id</code>	Stable step identifier used for execution feedback, dependency tracking, and possible replan joins.
<code>steps[].name</code>	Skill name selected from the allowed skill registry. The planner should not invent skills or use robot-specific APIs directly.
<code>steps[].args</code>	Skill arguments. They should refer to known entity IDs, explicit primitive values, or arguments supported by the selected skill schema.
<code>steps[].requires</code>	Dependencies between steps, if any. This allows the runtime to know which steps depend on previous successful outcomes.
<code>steps[].on_failure</code>	Recovery policy proposed by <code>planner_llm</code> for this step. If the step fails, <code>nao_orchestrator</code> may fail safely, continue if allowed, request user input, or send feedback back to <code>planner_llm</code> for replanning. Allowed values include <code>replan</code> , <code>clarify</code> , <code>ask_user</code> , <code>continue</code> , <code>ignore</code> , and <code>fail</code> .

Runtime recovery metadata. The current contract supports recovery metadata as skill failure may require controlled recovery. Each step may carry a `retry_budget`, and the plan envelope may also carry a plan-level `retry_budget`. This does not mean that `retry` is a separate `on_failure` value. Instead, retry behaviour is controlled by `nao_orchestrator` using the available retry budget together with the step’s `on_failure` policy. The plan envelope may also carry `communication_policy`. This controls when the planner/dialogue path should communicate progress, completion, clarification needs, or failure.

When a step fails with `on_failure = replan`, `nao_orchestrator` halts or blocks continuation, publishes execution feedback to `planner_llm`, and waits for a revised validated plan before dispatching further actions. This keeps execution authority inside `nao_orchestrator`, while allowing recovery to be driven by feedback-driven planning rather than by hard-coded local replanning.

These fields are included because they support runtime recovery, traceability, and human-robot interaction, but the essential planner output remains the ordered `steps` list. If the planner cannot produce a plan, the output should represent that as a planning result rather than filling

the plan with ambiguous failure fields. For the simple version, **steps** is the most important field.

4.10 Contract 6: Skill Execution Result

Every dispatched skill returns a typed result before the orchestrator can mark its step as completed. The result distinguishes the execution status from the evidence that supports it. For state-changing skills, the evidence may also contain explicit KB effects. These effects are proposed by the skill that observed or produced the change; they are not inferred from the planner's expected-effects text in Contract 4.

```

1 {
2   "skill": "place_object",
3   "status": "succeeded",
4   "summary_text": "I placed cup_1 on table_1.",
5   "evidence": {
6     "object_id": "cup_1",
7     "destination_id": "table_1",
8     "kb_effects": [
9       {"operation": "remove", "statement": "robot oro:holds cup_1"},
10      {"operation": "add", "statement": "cup_1 oro:isOn table_1"}
11    ]
12  },
13  "failure": null,
14  "metadata": {"fake": true}
15 }
```

The orchestrator applies `evidence.kb_effects` only after a successful result and routes the statements through `kb_skills`. Removal effects clear stale holding or location relations before replacement facts are added. When the query seam is available, the orchestrator verifies that removed statements no longer resolve and that added or revised statements do. A failed mutation or post-condition check converts the step into a failure and is propagated through Contract 7; later dialogue therefore cannot treat an unverified material change as current KB truth. Real and fake skills use the same result shape, although the physical evidence available to them differs.

4.11 Contract 7: Execution Feedback

For the simple version, execution feedback can be minimal. The planner needs to know which goal is affected, which step failed or succeeded, and a compact result if that result is useful for replanning or dialogue. A larger payload is justified only when the goal-state helper, orchestrator, traces, or debug tooling use the extra fields.

```

1 {
2   "goal_id": "goal_turn_123",
3   "status": "failed",
4   "step": {
5     "id": "step_1",
6     "name": "look_at"
7   },
8   "reason": "target frame unavailable",
9   "result_payload": {
10    "target_found": false,
11    "target_kind": "person"
12  }
13 }

```

Runtime feedback may include additional fields used by `nao_orchestrator`, logging, and the goal-state helper. Lineage uses `plan_id` and `plan_version`; event handling may add `event_type`, `validation_status`, `blocking`, and `needs_user_input`; observability may add `timestamp_sec`, `result_summary`, and retry metadata. These fields support execution control and debugging, while the minimal planner-facing feedback remains the goal, step, status, reason, and result payload.

4.12 Contract 8: Planner Dialogue Act

Planner dialogue acts represent planner intent to communicate, not direct speech execution. They are relayed through the orchestrator and realized by the dialogue layer.

```

1 {
2   "goal_id": "goal_turn_123",
3   "act": "ask_clarification",
4   "await_user_response": true,
5   "reason": "multiple visible people",
6   "text_hint": "Which person should I look at?",
7   "slots_needed": ["target_person"]
8 }

```

Allowed dialogue acts are acknowledgement, progress update, clarification, help request, failure explanation, completion notification, and cancellation notification. The planner does not speak directly; it emits a structured communication request.

4.13 Contract 9: Raw Evidence

Raw evidence is an ownership boundary rather than one universal message. Detector observations and `/scene/summary` remain with `nao_scene_grounding`; KnowledgeCore query rows

remain behind `kb_skills`; robot and simulated action evidence remains in Contract 6 skill results; and structured trace events preserve the corresponding route, plan, and step identifiers. Compact grounding in Contract 3 and planner feedback in Contract 7 are projections of this evidence for specific consumers. They do not replace the underlying source payloads.

Chapter 5

Implementation: ROS Nodes and APIs

This chapter describes how the architecture introduced in Chapter 3 and the contracts defined in Chapter 4 are realised in the ROS 2 workspace. The implementation deliberately separates semantic interpretation, planning, deterministic execution, perception grounding, symbolic knowledge, and skills. LLM components can therefore interpret or propose actions, but physical effects remain mediated by typed ROS interfaces and validated skill contracts.

The workspace combines locally developed integration packages with upstream-aligned ROS4HRI components. In particular, `dialogue_manager` provides a bridge between robot and user interaction, while the `chatbot_llm` package supplies intent extraction, response generation, and the turn-processing interface extended for this system. The planner-facing contracts, orchestrator, perception pipeline, fake-skill infrastructure, skill registry and NAO adapters are integrated in the main `nao-ros4hri-bridge` workspace. This distinction matters: the contribution is not a replacement dialogue framework, but a planner-mediated execution architecture fitted to existing ROS4HRI seams.

5.1 Runtime Flow

A user utterance first enters `dialogue_manager`, which sends a `DialogueInteraction` request to `chatbot_llm` with the dialogue state required for the turn. The chatbot combines this state with a symbolic representation of the current scene derived from the knowledge base, performs the call to the LLM for intent classification and response generation, and classifies the result as `dialogue`, `knowledge_query`, or `execution`. `dialogue` and `knowledge_query` turns return a response to the dialogue owner without entering the planner. `execution` turns additionally produce a structured planner request with goal metadata, normalised intent hints, dialogue history and current world state. This request is published to `/nao_orchestrator/planner_request` for admission.

Candidate requests are published to the planner-admission endpoint defined by Contract 2 (Table 4.3). This endpoint is owned by `nao_orchestrator`, which normalises the payload, establishes goal lineage, and forwards only admissible requests on `/planner/request`.

`planner_llm` generates a structured plan from that request and the shared skill registry, including execution order, relevant KB entities such as objects, locations, and people, and the metadata required for deterministic dispatch. The resulting planner output returns to the orchestrator, where its schema, skill names, arguments, and step ordering are checked before any action or service is invoked. Step outcomes are published on `/planner/execution_feedback`. A goal-keyed planning-state helper inside `planner_llm` consumes this feedback when the orchestrator reports a relevant event, such as a failed step, a clarification requirement, or a recoverable condition. It updates the active planning state and may request a replan or planner dialogue act. The planner node itself does not monitor or control robot execution.

This flow enforces three runtime constraints from the requirements in Table 3.1. First, a dialogue-only turn cannot reach robot execution unless the chatbot selects the execution route and the orchestrator admits the resulting request (FR2–FR3). Second, planner output remains a candidate plan until its schema, skill names, arguments, failure policies, and lineage fields pass deterministic validation (FR5). Third, KB facts supplied with a planner request describe what the system knows when planning begins. They can support target selection, but they cannot prove that a later action succeeded or that the world remained unchanged; the responsible skill must provide fresh execution evidence before completion is reported.

Figure 5.1 presents the resulting node graph, python packages, ROS buses, external backends, validation seams, and extension points.

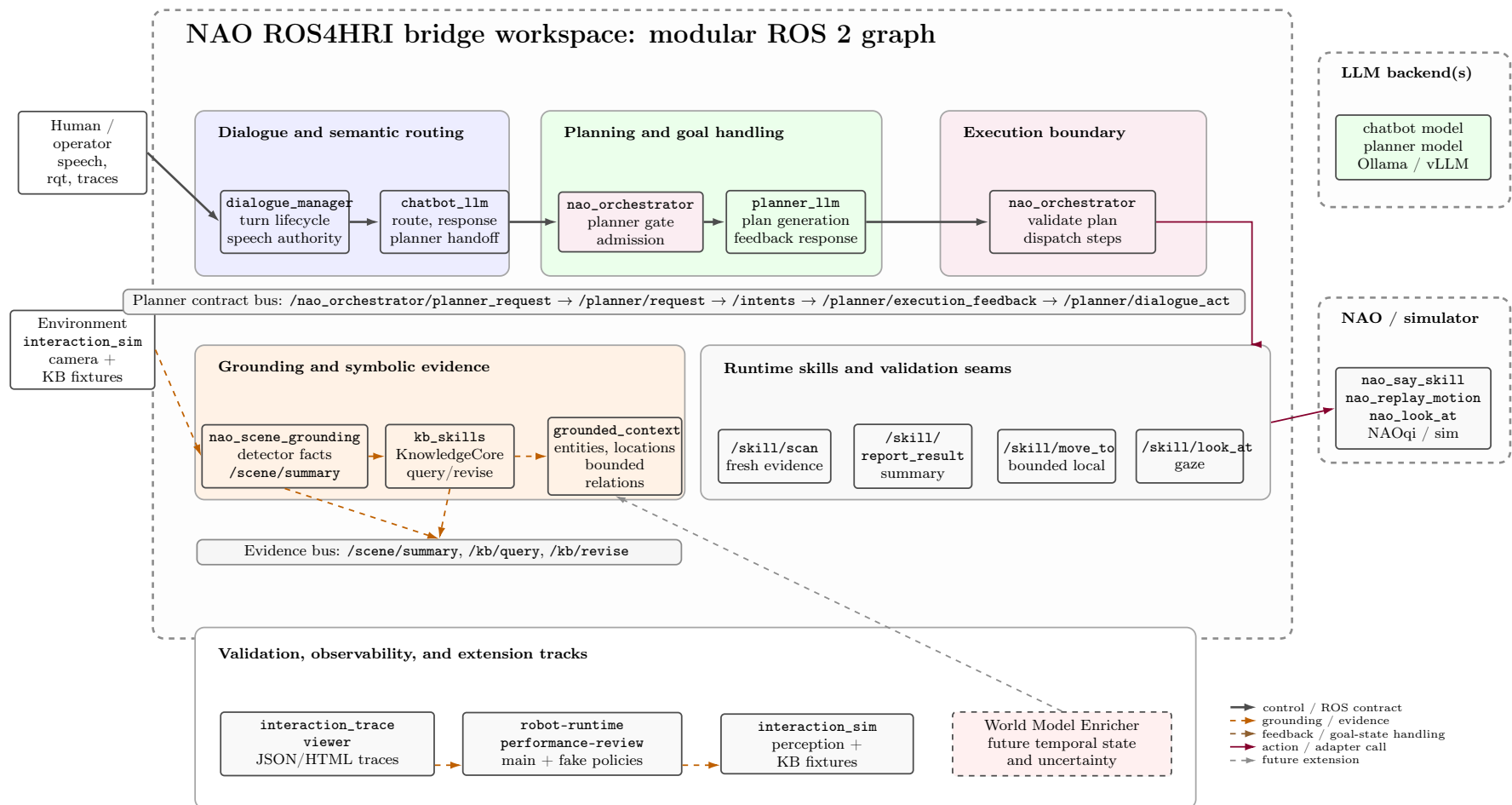


Figure 5.1: Full-stack implementation of the NAO ROS4HRI bridge. Dialogue components are blue, grounding and evidence components orange, planning components green, execution components purple, and skills or adapters neutral grey. The `interaction_sim` and KB-fixture paths provide controlled validation inputs. The dashed red World Model Enricher box denotes future work rather than an implemented runtime component.

5.2 ROS Nodes

5.2.1 `dialogue_manager`

`dialogue_manager` is the ROS 2 lifecycle component responsible for multi-modal communication between the robot and its interlocutors. Following the ROS4HRI package design¹, it tracks concurrent conversations per person and per interaction group, preserves chronological utterance histories, and associates incoming `hri_msgs/LiveSpeech` streams with the relevant dialogue. Group utterances are fanned out to the corresponding personal and group histories, while prior summaries can be restored when a known interlocutor begins a new session. The node invokes the chatbot through `DialogueInteraction`, exposes the high-level `/skill/chat`, `/skill/ask`, and `/skill/say` interfaces, and realises final user-facing expressions through the configured Say sub-skill. The planner integration therefore extends an existing multi-user dialogue lifecycle rather than replacing it.

This ownership boundary is also a runtime constraint: planning and execution packages may publish structured communication requests, but they do not call the speech backend directly merely because they have produced status information. This preserves the separation between dialogue and execution introduced by FR2 and the ownership map in Table 4.1.

Planner dialogue acts return through the same ownership boundary. `planner_llm` publishes structured acknowledgement, clarification, failure, or completion acts on `/planner/dialogue_act`; `nao_orchestrator` relays the admitted act on `/nao_orchestrator/planner_dialogue_act`; and the dialogue layer chooses the final wording and speech mechanism. This implementation of Contract 8 preserves one utterance authority per semantic event and makes duplicate acknowledgement or completion speech observable in the validation traces.

5.2.2 `chatbot_llm`

`chatbot_llm` is a lifecycle-managed semantic routing node built on the existing chatbot scaffold. Its turn callback receives a `DialogueInteraction` and the conversation history assembled by `dialogue_manager`. Before response generation or any planner handoff, it queries the KB through `kb_skills`, reads the most recent `/scene/summary`, and passes both sources through the `PlannerHandoff` projection helper. The result is the compact `grounded_context` defined by Contract 3 (Table 4.4 and Figure 4.1). It contains a bounded set of entities, locations, and task-relevant relations instead of raw detector streams or unrestricted KB rows. This is the principal KB-to-chatbot seam: the same projection informs dialogue and knowledge answers on every user turn, and is attached to Contract 2 only when the selected route produces a planner request.

The turn engine implements the route field from Contract 1 as follows:

- A `dialogue` route covers conversational and task-discussion turns and does not publish planner work.

¹Package source: https://github.com/ieverythng/dialogue_manager.

- A `knowledge_query` route answers from current symbolic evidence without causing a physical action. For example, “what can you see?” can use the current grounded context, whereas “scan and tell me what you see” requests fresh sensing and is eligible for execution.
- An `execution` route constructs planner-facing metadata such as goal text, request kind, normalised intent hints, and target KB entities. It publishes a candidate request to `/nao_orchestrator/planner_request`, but it does not create a skill sequence.

The final point implements FR3 and the admission ownership in Contract 2: `chatbot_llm` may propose execution, but only `nao_orchestrator` can admit the request and only a validated planner output can reach skill dispatch.

The canonical behavioural policy resides in `src/chatbot_llm/config/chat_prompt_pack.yaml`. Python builders provide structural framing and runtime context, while the chatbot sequence uses response and intent stages that share the same system prompt, route policy, skill catalogue, and grounded context. The baseline is `response_first`. In planner mode, the response-stage call returns the natural `verbal_ack` together with the route and confidence fields from Contract 1. The intent stage then defines the user intents and, for an execution route, constructs the candidate planner handoff from that staged result. The response stage therefore performs route selection as part of the same semantic decision that produces the acknowledgement; it does not create an executable plan. In the `intent_first` ablation, the intent stage runs first, its route is checked and locked, and the response stage generates wording under that locked route. The comparison tests whether early route commitment changes contradictions between the spoken response and planner admission. The proposed Intent–Route–Response (IRR) variant goes further by producing the three outputs atomically, but it remains a future or separately scored implementation until its trace and route-consistency behaviour are validated.

Every completed turn emits `/chatbot_llm/turn_trace`. The trace records the final route, intent source and confidence, grounded-context visibility, spoken acknowledgement, and whether a planner handoff was published. It therefore provides the observable link between Contract 1, Contract 2 admission, and the route-consistency evaluation in Chapter 6.

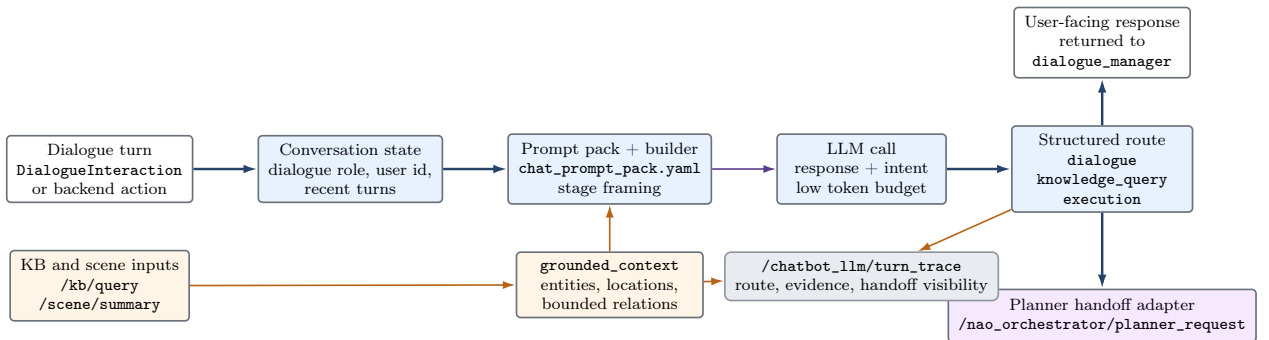


Figure 5.2: Internal `chatbot_llm` path. The node owns semantic interpretation and candidate planner metadata, but not plan construction, dispatch, or speech execution.

5.2.3 planner_llm

`planner_llm` is the high-level planning node and contains a goal-keyed planning-state helper for feedback-driven state transitions. The helper is a subprocess-like part of the node, not an independent execution supervisor. The node subscribes to admitted `/planner/request` messages rather than directly consuming chatbot output. Through `planner_common`, each request is normalised into the canonical contract with goal metadata nested under `plan`. The planner prompt combines the goal text, request kind, normalized intent hints, grounded context, the shared skill-entry projection from Contract 4, permitted step types, and the required JSON output envelope. Planner policy is maintained in `src/planner_llm/config/planner_prompt_pack.yaml`; the runtime builder supplies request-specific structure without duplicating that policy in Python source.

The planner prompt is assembled from the admitted request, bounded grounded entities, the skill registry fields relevant to selection and ordering, the permitted step types, and the planner output contract. The model is told which skills are available, which arguments are required, which effects are expected, which result evidence can support completion, and which failure modes may require clarification or replanning. This registry projection is the main link between the model’s language-level proposal and the typed ROS execution surface.

The model response is not treated as executable until the planner engine extracts the candidate object, normalises its schema, validates required metadata and step fields, and applies one bounded repair retry when configured. Unsupported skill names, malformed arguments, invalid recovery policies, or unparseable output become typed failure or clarification outcomes. The planner records the selected output mode, including a normal model plan or one of the explicitly named deterministic fallback modes described in Section 5.3. Valid plans are published on `/intents` for independent orchestrator validation.

The node also subscribes to `/planner/execution_feedback`. The embedded planning-state helper associates relevant events with the active goal and asks the planning path to respond when the orchestrator reports completion, failure, a recoverable condition, or a need for user input. That response may be a revised plan or a structured dialogue act; it is not robot control. The planner never invokes a robot adapter and never speaks directly. Its user-facing output must cross the orchestrator relay and dialogue owner.

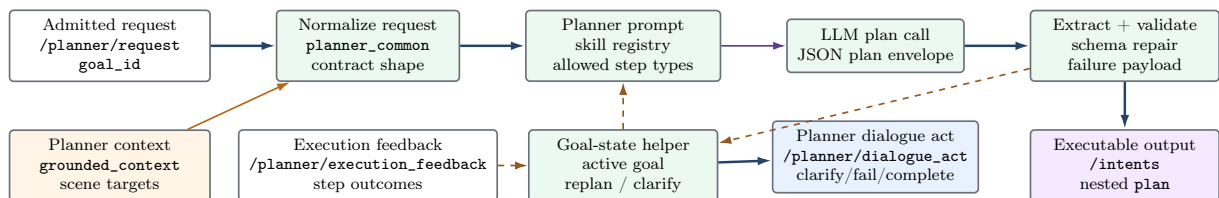


Figure 5.3: Internal `planner_llm` path, from admitted request through validated plan envelope and feedback-driven goal-state handling.

5.2.4 Goal-Keyed Planning-State Helper

The goal-keyed planning-state helper maintains planning state under an active `goal_id`; it does not monitor or control robot execution. Each accepted plan additionally carries a `plan_id` and monotonically meaningful `plan_version`; stable `step_id` values allow a newer version to preserve already completed work when a replan joins the same goal. Feedback from an older or unrelated version is rejected rather than being allowed to complete the current goal accidentally.

The helper distinguishes planning, replanning, waiting-for-user, completed, and failed states. Step feedback may move the goal forward, expose a recoverable condition, or produce a structured dialogue act. If a replacement plan is necessary, `planner_llm` creates and validates it under a newer version. `nao_orchestrator` never invents a local substitute plan, which preserves a single planning authority while keeping execution deterministic. The helper maintains order and lineage across these transitions; it does not supervise the robot or replace the orchestrator’s execution checks.

5.2.5 Skill Registry (`skill_common`)

The planner-facing capability model is derived from the canonical skill registry maintained by `skill_common`. Contract 4 and Table 4.5 define the shared entry shape: canonical name and aliases, typed parameters, preconditions, expected effects, observable success evidence, failure modes, result schema, safety flags, runtime availability, and executor mapping. The planner receives a bounded projection of these fields when it selects and orders skills. The orchestrator uses the same names, argument requirements, and availability information when it validates Contract 5 plans and dispatches Contract 6 calls. This arrangement provides a machine-checkable guard against prompt and executor drift. The final scored source revision is accepted only when the generated planner and documentation projections pass the registry consistency checks against the canonical entries.

The registry also separates semantic availability from the adapter selected by a launch profile. A skill may be implemented by a first-party robot action, a deterministic fake server, or both. The profile must still prove that the selected endpoint is running before execution. Table 5.1 presents the complete planner-visible surface in a compact form; the per-skill argument, effect, failure, and executor fields are expanded in Appendix A.1.

Table 5.1: Planner-visible runtime skills and their current execution profiles.

Skill(s)	Contract role	Current adapter/profile	Fake validation
<code>scan</code>	Refresh scene evidence and return a typed scan payload.	First-party <code>/skill/scan</code> server.	No separate fake route.
<code>find_object</code> , <code>inspect_area</code>	Return object, person, or area evidence with typed ambiguity and backend failures.	Deterministic fake server in the evaluated profile.	Yes.
<code>navigate_to</code>	Reach a semantic destination such as a person, object, or location.	Deterministic fake navigation endpoint.	Yes.
<code>walk_to</code>	Execute a short body-relative locomotion segment.	Real <code>/skill/move_to</code> or fake endpoint, selected by profile.	Yes.
<code>pick_object</code> , <code>place_object</code> , <code>bring_object</code>	Change object holding, support, and delivery relations and return KB effects.	Deterministic manipulation servers in the current evaluated stack.	Yes.
<code>perform_motion</code> , <code>look_at</code>	Change posture, head orientation, or gaze target.	First-party replay, head-motion, and look-at adapters; fake endpoints are selectable in simulation.	Yes.
<code>wave_greet</code>	Perform a guarded social greeting gesture.	Real replay-motion gesture or fake endpoint.	Yes.
<code>kb_add</code> , <code>kb_revise</code> , <code>kb_remove</code>	Apply explicit symbolic mutations through KnowledgeCore.	First-party orchestrator to <code>kb_skills</code> path.	No hardware fake required.
<code>ask_user</code> , <code>report_result</code>	Request missing information or report evidence-derived execution results.	First-party dialogue relay and report/say skill servers.	Trace-driven.

5.2.6 `nao_orchestrator` and the Execution Boundary

`nao_orchestrator` is the deterministic boundary between executable intent and robot effect. It subscribes to `/intents` for both direct and planner-generated work. A direct intent is normalised, checked against duplicate and argument guards, and mapped to a registered execution path. When the intent is an explicit speech operation such as `say` or `greet`, dispatch is additionally controlled by the profile’s `dispatch_speech_intents` setting before the `/nao/say` action is called. Ordinary conversational responses do not cross this path and remain under `dialogue_manager`; `report_result` is an execution skill whose evidence-derived text is forwarded through the report and say skill servers.

Planner-mediated work enters through a second ingress. The planner gate admits the Contract 2 request, normalises request kind and goal lineage, stores bounded request context, and publishes `/planner/request`. The returned Contract 5 plan arrives on `/intents`; the orchestrator checks its nested metadata, ordered steps, registry names, arguments, dependencies, and failure policies against Contract 4 before publishing `plan_accepted`. This two-sided role explains why the node appears at both admission and execution positions in Figure 5.1: it is one lifecycle component enforcing two interfaces, not two planning nodes.

Accepted plans are dispatched in order through ROS actions or skill-server interfaces appropriate to each step. The orchestrator retains the active goal, plan version, current step, completed results, and pending work. It publishes `step_started`, `step_succeeded`, `step_failed`, and `plan_completed` events through Contract 7, preserving `goal_id`, `plan_id`, `plan_version`, and `step_id`. This continuous execution state makes a run reconstructable and allows the

planning-state helper to reject stale callbacks or respond to a recoverable failure without moving robot control into the planner.

On failure, the orchestrator reports the typed Contract 6 result and stops, continues, or waits according to the validated policy. It does not ask an LLM for an unregistered action, create a replacement plan, or fabricate success. Planner dialogue acts cross the Contract 8 relay and remain under dialogue-layer wording and speech ownership. The registered `report_result` path is the explicit execution-speech case: the orchestrator derives its content from Contract 6 evidence and dispatches it through the report and say skill servers. These restrictions implement FR2, FR3, FR5, FR6, and FR7 by keeping route ownership, planning, execution, evidence, and speech authority at their declared seams.

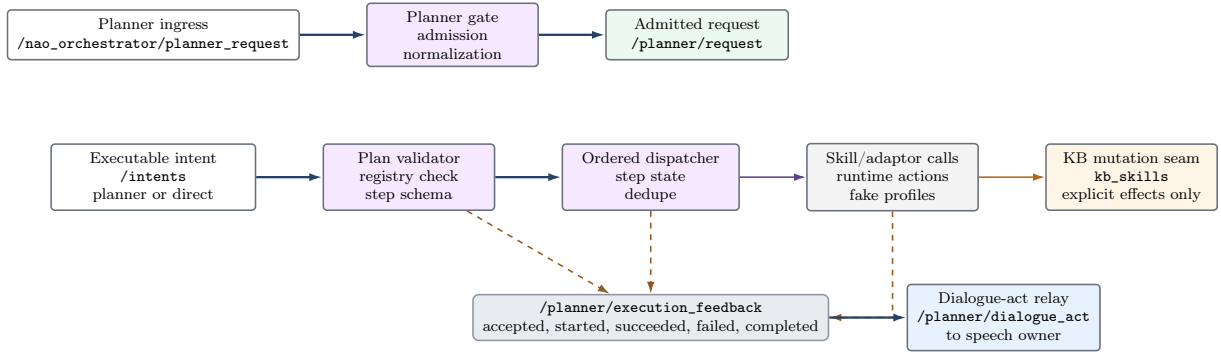


Figure 5.4: The `nao_orchestrator` admission and execution boundaries. All effects cross deterministic validation and retain goal and plan lineage.

5.3 Deterministic Guards and Bounded Fallbacks

The normal path is model-assisted, but several deterministic guards constrain the interfaces around it. These guards implement the admission and evidence requirements in FR2, FR3, FR5, FR6, and FR7. They are deliberately narrow and observable. A fallback does not create a second hidden planner, and it cannot convert missing evidence into a successful result.

At the chatbot boundary, `chatbot_llm` normalizes the route produced by the response and intent stages and checks for contradictions between the route, acknowledgement, and extracted intent. A missing or contradictory response can be repaired with a constrained route decision. When planner mode is disabled, the rules path can answer supported non-execution turns; when planner mode is enabled and the language-model result cannot be repaired safely, the node returns a typed failure rather than fabricating an execution request. In the `intent_first` mode, the locked route is carried into response generation and execution wording is sanitized against that route. These guards enforce the route and ownership boundaries in Contracts 1 and 2.

At the planner boundary, the engine first attempts the configured model path, extracts and validates the candidate plan, and performs at most the bounded validation retry configured by the planner prompt pack. If the candidate remains invalid, the decision is marked as a

planner failure or clarification. Three narrow deterministic modes are retained for cases whose scope is explicit in the source and trace: `rule_fallback` handles one unambiguous primitive motion or posture request without composite markers, target ambiguity, or a requested report; `grounded_location_group_fallback` handles a grounded request to bring every object in a known location to a known recipient or destination and creates one delivery step per grounded object; and `grounded_look_fallback` handles a grounded look-at or gaze request, optionally followed by evidence-based reporting. Each mode is recorded in the planner decision metadata and remains subject to the Contract 4 skill surface and Contract 5 plan validation. Composite or target-ambiguous language is not silently reduced to an incomplete plan.

At the execution boundary, `nao_orchestrator` applies deterministic skill aliases, argument checks, lineage checks, stale or duplicate-event guards, and the selected failure policy before dispatch. Its `report_result` path derives the final summary from successful result payloads or trace evidence, so the planner cannot predeclare an effect that the skill did not observe. Profile-dependent adapter fallbacks, such as real versus fake motion endpoints, remain execution configuration and do not change planner policy. Skill results follow Contract 6, feedback follows Contract 7, and any clarification or completion request crosses Contract 8 and the dialogue owner before speech.

5.4 Grounding and Knowledge

`nao_scene_grounding` converts detector output into a provider-independent scene representation. The evaluated robot/RViz profile selects `emorobcare_cv`, whose packaged custom YOLO detector was trained for a restricted proof-of-concept object set and integrated with the NAO camera stream through ROS. Repository documentation identifies blueberry, corn, pear, tomato, and zucchini as its strongest demonstration classes. The launch layer also exposes `yolo_ros`; its default `object_detection_model` is `yolov8n.pt`, but this fallback was not the primary evaluated detector. Both backends are normalized into the same internal observation shape, so downstream nodes do not depend on either detector message type.

The grounding node filters detections by confidence and an explicit label allowlist, maps labels to KB classes, separates people from objects, and maintains local identifiers using tracker IDs or bounded image-plane matching and recency rules. It publishes `/scene/summary` and mirrors accepted observations as transient KnowledgeCore statements through `/kb/revise`. These statements include class, visibility, detection source, score, visual centre, and last-seen time. Their finite lifespan prevents an old camera observation from becoming permanent world state. This implements the evidence ownership in Contract 9, while Contract 3 exposes only the bounded symbolic projection needed by dialogue and planning.

`kb_skills` is the transport boundary for `/kb/query` and `/kb/revise`. The chatbot obtains a bounded knowledge snapshot through this interface and combines it with the current scene summary to produce `grounded_context`. This projection deliberately omits unrestricted raw evidence and keeps entities, locations, and relations within prompt-size bounds. Image-plane centres are retained as image evidence but are not interpreted as metric distance. Metric

position or proximity requires a frame-qualified spatial overlay or fresh evidence from the responsible skill.

Planning context and execution evidence consequently have different roles. Current symbolic state is sufficient for a non-mutating knowledge answer and useful for selecting a plan. It is not sufficient to claim that an object was found after a scan, that a path remains clear, or that a manipulation succeeded. A skill therefore returns a Contract 6 payload containing the evidence it observed and, when it materially changes a KB entity, explicit `evidence.kb_effects`. Successful fake `pick_object`, `place_object`, and `bring_object` results already remove stale holding or source-location relations and add the new support, destination, or recipient relation. The orchestrator applies those effects through `kb_skills` and verifies their post-conditions through `/kb/query` when available. A failed update or verification fails the step rather than allowing later dialogue to report the intended effect as current state. Real manipulation adapters can adopt the same payload without changing planner or KB ownership.

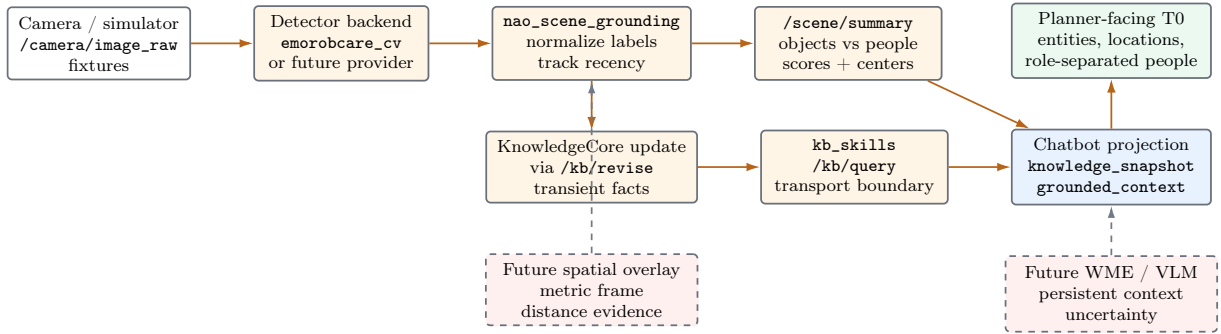


Figure 5.5: Combined perception and symbolic-grounding pipeline. Compact projections serve dialogue and planning, while raw and fresh evidence remains at its owning seam.

5.5 Fake Skills and Simulation Seams

The `fake_skills` package supplies deterministic action servers for the validation routes identified in Table 5.1. It preserves the Contract 6 result shape while replacing hardware execution with an explicit policy. The server resolves request-level overrides, named scenario settings, per-skill overrides, or a global `scenario`, `always_success`, `always_fail`, `every_other`, or seeded pseudo-random mode. Each result records `metadata.fake=true`, the scenario and result mode, its policy source, deterministic seed, and simulated duration. Dispatch and outcome events are also published on `/fake_skills/events`.

This server lets the same planner and orchestrator path exercise all-success, fail-once, blocked-delivery, missing-recipient, and ambiguous-evidence cases. Successful manipulation results can emit the KB effects described above, so later steps and turns observe a changed scenario state rather than a fixed textual fixture. The fake package does not duplicate the planner’s skill catalogue; it implements selected Contract 4 entries behind the executor mappings chosen by the launch profile.

The resulting evidence is intentionally qualified. A fake-skill pass supports claims about routing, plan validity, deterministic dispatch, feedback handling, and recovery policy. It does not demonstrate physical navigation accuracy, grasp success, detector quality, NAOqi timing, or human-robot safety. Chapter 6 therefore reports the multi-step deterministic fake-skill score separately from the main speech-ingress stack and identifies robot- or detector-dependent checks explicitly.

5.6 Models and Runtime Configuration

The integrated runtime is exposed through two principal `nao_chatbot` launch profiles. Both compose dialogue, chatbot, planner, orchestration, grounding, skill servers, and observability arguments through the same launch builder, but they select different perception and embodiment sources.

Table 5.2: Principal integrated launch profiles used by the implementation.

Launch profile	Default runtime source	Primary use
<code>nao_chatbot_sim.launch.py</code>	<code>interaction_sim</code> perception and tools, deterministic fake skills, simulated gaze by default	Dialogue, planner, grounding, ordered dispatch, failure injection, and preloaded-environment validation without requiring the robot.
<code>nao_chatbot_robot.launch.py</code>	Packaged NAO robot description and TF tree, robot camera, HRI visualization, RViz, detector and scene grounding	Robot-camera and frame validation, real gaze and motion adapters, detector-backed KB entities, and guarded hardware execution.

The principal launch controls are grouped by profile:

- The simulator profile exposes `start_interaction_sim`, `start_interaction_sim_perception`, `start_interaction_sim_tools`, `start_fake_skills`, `preloaded_environment_ids`, and per-skill execution modes.
- The robot/RViz profile exposes `start_ nao_robot`, `start_rviz`, `start_object_detection`, `object_detection_backend`, and `start_scene_grounding`. Simulator tools may remain active while simulator perception is disabled, preventing synthetic camera data from competing with the robot stream.

Both profiles expose `chatbot_turn_pipeline_mode`, the grounded-context digest switch, detector thresholds, timeouts, and trace-viewer channels as launch arguments rather than source edits.

The current lab defaults use `QuantTrio/Qwen3-VL-30B-A3B-Instruct-AWQ` for both chatbot and planner inference through an OpenAI-compatible vLLM endpoint, with model reasoning disabled for the scored path. The planner default uses temperature 0.1 and an 800-token output bound; the chatbot uses separate response and intent token limits. The same launch arguments can select an Ollama provider or different models without changing node contracts.

Credentials are read from a named environment variable, while the endpoint class, model identifiers, temperatures, token limits, timeouts, and hardware are frozen in each validation manifest.

Lifecycle sequencing is part of the runtime configuration. The launch builder configures and activates `chatbot_llm`, `dialogue_manager`, `nao_orchestrator`, and the robot-facing `say`, `scan`, and `report-result` servers before declaring the dialogue path ready. `planner_llm` is a standard ROS node launched with provider preflight checks and explicit failure visibility. Final validation also records the selected response mode, active real or fake adapters, detector backend, environment fixture, and launch arguments. These facts form the reproducibility bundle defined in Chapter 6.

Chapter 6

Validation Methodology

6.1 Evaluation Aim

The validation methodology is designed to test the thesis claim at the architectural level: a robot-facing LLM planner is useful only if its plans remain grounded in the live ROS4HRI state, respect the available skill contracts, expose execution failures through typed feedback, and produce user-facing dialogue through the intended speech owner. The evaluation therefore does not treat success as a single demonstration outcome. It treats success as a combination of route correctness, plan validity, execution validation, recovery behaviour, and traceability.

The strategy separates four evidence layers so that a success in one layer cannot conceal a failure in another. First, controlled planner-request probes use fixed grounded context to isolate schema compliance, skill selection, plan lineage, and orchestrator dispatch from speech and dialogue timing. Second, the thesis-facing questionnaire injects complete user turns through the ROS4HRI speech path in the canonical **response_first** profile; this is the main full-stack score because it covers routing, planner handoff, execution feedback, and exactly-once speech. Third, named environment fixtures preload deterministic KnowledgeCore and scene state so that grounding-sensitive cases can be repeated. Fourth, a separate multi-step deterministic fake-skill policy ladder injects staged skill outcomes to test success, recoverable failure, ambiguity, and replan behaviour without depending on physical hardware. The **intent_first** mode and a detector-enabled launch are reported as ablation and perception profiles respectively, not merged into the baseline score.

6.2 Validation Goals

Table 6.1 gives the validation goals used to score the system. These goals are deliberately operational. Each goal can be checked from ROS topic traces, planner JSON, fake-skill events, or container logs.

Goal	Question	Evidence source
Route correctness	Does the chatbot keep dialogue-only turns out of the planner and send execution-looking turns to the planner or orchestrator?	<code>/chatbot_llm/turn_trace</code> , <code>/intents</code> , <code>/planner/request</code>
Plan validity	Does the planner output valid JSON using admissible skill names, argument fields, and failure policies?	Planner output, <code>planner_common</code> normalisation, orchestrator validation
Skill coverage	Are all requested subgoals represented in the emitted plan, including composite instructions?	Plan steps, expected-plan checklist
Grounding use	Does the planner use <code>grounded_context</code> , <code>knowledge_snapshot</code> , <code>scene_summary</code> , and <code>state_t0</code> when a task depends on visible objects or people?	Planner request payload, KB query evidence, <code>/scene/summary</code>
Execution validation	Does <code>nao_orchestrator</code> execute only validated steps and return structured feedback to the planner?	<code>/planner/execution_feedback</code> , orchestrator logs
Failure handling	Does a failure cause clarification, replanning, or safe termination rather than a false success message?	Fake-skill result payloads, planner dialogue acts
Exactly-once speech	Does one semantic event produce one user-facing response, without duplicated acknowledgement or completion speech?	<code>/planner/dialogue_act</code> , <code>/debug/nao_say/speech</code> , dialogue manager output
Trace completeness	Can each run be reconstructed from the saved JSONL trace and report bundle?	<code>interaction_trace_viewer</code> , <code>/fake_skills/events</code> , validation reports
Phase completeness	Were ingress, route, planner request, execution feedback, terminal state, and speech actually observed rather than inferred from a timeout fallback?	Per-case <code>phase_observations</code> , fallback markers, timestamps
State freshness	Did the case use the declared fixture and avoid stale facts from a previous run?	Fixture manifest, KB evidence, stale-world guard

Goal	Question	Evidence source
------	----------	-----------------

Table 6.1: Validation goals for planner-mediated ROS4HRI execution.

6.3 Questionnaire-Based Runtime Checks

The runtime questionnaire complements the structured validation suite. Its purpose is to expose integration faults that planner-only tests can miss. Each item is marked as *pass*, *degraded*, *fail*, or *not run*. A failure in simple dialogue, duplicate speech, raw planner leakage, or false execution success is treated as a high-severity result because it directly affects user-visible behaviour.

Category	Representative prompt	Expected behaviour
Simple dialogue	“Hey”, “How are you?”, “What is your favourite colour?”	The route remains dialogue-only, no planner request is emitted, and exactly one response is spoken.
Knowledge dialogue	“What can you see?”, followed by a fixture or detector-state change and a repeated query	The declared environment state appears in <code>grounded_context</code> ; stale entities from a previous case do not contaminate the answer.
Simple skill	“Move your head to the right”, “Wave at me”	The system emits one acknowledgement, dispatches one skill, and reports truthful completion or failure.
Composite skill	“Navigate to the phone and tell me what else you see”	The planner emits ordered steps, preserves intermediate evidence, and reports the complete chain.
Simple fake skill	<code>find_object(cup)</code> under success, ambiguity, and failure modes	Fake payloads include <code>metadata.fake=true</code> , stable result modes, and recoverable failure fields where appropriate.
Composite fake skill	Multi-step navigation, search, and report under controlled fake outcomes	Completed steps are not repeated unnecessarily, failed steps are explained, and clarification is routed through dialogue.

Table 6.2: Tailored runtime questionnaire used for end-to-end validation.

6.4 Fake-Skill Scenario Validation

The fake-skill suite is the deterministic substrate for recovery testing. It avoids the main weakness of prompt-only simulation: the planner and orchestrator still interact with ROS

actions that return typed payloads, emit `/fake_skills/events`, and can update controlled scenario state. The package supports global modes such as `scenario`, `always_success`, `always_fail`, `every_other`, and `random_seeded`, together with named or dynamic policies that fail a selected navigation, search, manipulation, delivery, or recipient-resolution stage.

The scenario suite is organised around case families rather than isolated commands. This makes it possible to compare planner behaviour under different outcome policies while keeping the user goal fixed.

Case family	Skill path	Outcome policies	Expected response
Object search	<code>find_object</code>	Found, not found, ambiguous, backend unavailable	Report object evidence, ask clarification when ambiguous, or explain failure.
Navigation	<code>navigate_to</code> <code>walk_to</code>	or Success, path blocked, unknown location, timeout	Continue on success, ask for alternative or safe stop on recoverable failure.
Perception and reporting	<code>scan -></code> <code>report_result</code>	Success, empty scene, backend failure	Fill report from payload when available and avoid unsupported claims.
Social gesture	<code>look_at -></code> <code>wave_greet</code>	Success, motion unavailable, safety disabled	Execute only admissible steps and explain social-motion failure.
Area inspection	<code>inspect_area</code> or <code>navigate_to</code> <code>-> scan -></code> <code>report_result</code>	Clear, object found, person found, ambiguous area	Report observed area state or request clarification.
Composite instruction	Ordered mix of navigation, search, and reporting	All success, first failure, middle failure, alternating failure	Preserve step order, stop or replan safely, and summarise completed work.

Table 6.3: Scenario matrix for deterministic fake-skill validation.

6.5 Multi-Step Deterministic Fake-Skill Scenarios

These scenarios combine multi-step fake execution, structured fixture state, deterministic failure injection, and trace-level evidence. They are important because many planning failures only appear after the first step has succeeded and the stack must decide whether to continue, replan, ask the user, or stop.

The deep suite uses a policy ladder over representative multi-step goals:

1. **All-success baseline.** Every planned step succeeds. The expected trace contains ordered, non-duplicated dispatch, complete lineage, one terminal state, and one final user-facing report.
2. **Fail-once navigation.** The first relevant navigation step fails recoverably and a later attempt may succeed. The planner must preserve completed work, issue a newer plan version when replanning, and avoid replaying unrelated steps.
3. **Fail-once manipulation.** A pick or placement stage fails after earlier movement or search has succeeded. The response must not claim delivery, and any retry must remain tied to the same goal with stable step lineage where applicable.
4. **Blocked delivery or missing recipient.** A required path, object, or recipient cannot be resolved. The expected outcome is clarification or truthful safe termination rather than an invented substitute or false completion.
5. **Seeded stress profile.** After deterministic policies pass, `random_seeded` explores additional outcome combinations while retaining an exact seed and event trace for reproduction.

6.6 Execution Profiles

The primary thesis-facing mode is full user-turn injection through the ROS4HRI speech ingress. Before a `LiveSpeech` event is published, the corresponding speaker is made visible on the tracked-voices stream using reliable, transient-local delivery. Scored questionnaire cases use a group voice scope so that the dialogue stack observes the same ingress contract as a live interaction. This profile is slower than direct request injection but is the only one that jointly tests chatbot routing, planner admission, feedback relay, and speech ownership.

Planner-request injection remains a diagnostic profile. A controlled payload with fixed `goal_text`, normalised intents, scene targets, and `grounded_context` is published at the planner gate to isolate the planner, orchestrator, and selected skill server. It is useful for narrowing a failure, but it does not receive the main end-to-end score. The multi-step deterministic fake-skill score is also reported separately because it measures goal handling under controlled action outcomes rather than robot hardware performance. Finally, object-recognition cases use a distinct detector-enabled profile when the main runtime deliberately launches with object detection disabled; a missing detector process is never counted as negative detector evidence.

6.7 Metrics

For each run, the validation log records the case id, utterance, route mode, fake-skill mode, scenario id, seed, planner call status, plan steps, dispatched skills, result statuses, clarification

flag, final status, final response text, timeout flag, warning count, and error count. The main aggregate metrics are:

- route accuracy;
- valid-plan rate;
- expected-skill coverage;
- fake-skill dispatch rate;
- safe-failure rate;
- clarification rate when ambiguity is expected;
- timeout rate;
- forbidden-plan rate, especially executable **say** steps when speech should remain under dialogue ownership;
- phase-completion rate across turn injection, route observation, planner request, execution feedback, terminal state, and speech;
- fallback-event rate, distinguishing observed ROS evidence from timeout or log-derived inference;
- stale-world contamination rate across fixture changes;
- exactly-once acknowledgement and terminal-speech rate;
- replan-lineage correctness, including plan-version changes and preservation of completed work;
- mean and maximum latency, when timestamps are available.

6.8 Evidence Bundle

A complete validation run should produce a reproducible bundle containing the suite configuration, JSONL trace, per-case metrics, aggregate metrics, and a short report. The expected layout is:

Listing 6.1: Expected validation evidence bundle.

```

1 docs/evaluation/runs/<timestamp>/
2   suite_config.yaml
3   runtime_metadata.json
4   environment_fixture.yaml
5   trace.jsonl
6   per_case_metrics.csv
7   aggregate_metrics.json
8   report.md
9   report.html
10  selected_traces/
11    <case_id>_trace.json

```

Each case trace records explicit `phase_observations`, fallback markers, a terminal `plan_outcome_summary`

planning and KB evidence, and the speech events used for exactly-once assessment. Runtime metadata identifies the source revision, launch profile, model configuration, prompt-pack revisions, registry projection, fixture, and fake policy or physical adapter. The stale-world guard records whether undeclared state from an earlier case was observed. The thesis uses this structure as the evidence contract; focused probes remain diagnostic until reproduced by the declared suite and profile.

Chapter 7

Results

7.1 Current Status of the Validation Evidence

The results in this chapter should be read as the first populated validation pass rather than as a final statistical evaluation. At the time of this thesis draft, the available evidence supports three claims. First, the core ROS stack can launch with the planner, chatbot, orchestrator, trace viewer, and fake-skill server active in the same profile. Second, deterministic fake-skill scenarios can return the expected structured failure modes for ambiguity, blocked navigation, and unavailable social motion. Third, the live user-turn and planner-admission paths still show integration risks that must be resolved before the final validation suite can be treated as complete.

7.2 Runtime Stack Health

A live-stack validation pass on 26 May 2026 used the `nao_ros2` container with fake skills and the interaction trace viewer enabled. The planner, chatbot, orchestrator, trace viewer, and fake skill server were observed as active. The trace viewer was subscribed to the key planning and execution topics, including `/planner/request`, `/intents`, `/planner/execution_feedback`, `/planner/dialogue_act`, `/chatbot_llm/turn_trace`, and `/fake_skills/events`.

Runtime component	Observed status	Evidence note
planner_llm	Ready	Planner startup reported readiness with automatic replanning enabled.
chatbot_llm	Ready	Chatbot startup reported planner mode enabled.
nao_orchestrator	Active	Orchestrator was running as the deterministic execution boundary.
interaction_trace_viewer	Active	Subscribed to planner, intent, feedback, dialogue, chatbot trace, and fake-skill event topics.
fake_skill_server	Active	Named scenarios were available, including <code>ambiguous_cup</code> , <code>area_person_found</code> , <code>path_blocked</code> , and <code>social_wave_unavailable</code> .

Table 7.1: Observed runtime health during the live-stack validation pass.

7.3 Action-Level Fake-Skill Results

The strongest current result is the deterministic behaviour of the fake-skill action path. Three failure-oriented scenarios were tested directly at action level. Each returned the expected aborted status and failure code. This is important because the thesis validation methodology depends on being able to inject controlled execution outcomes without relying on physical robot variability.

Skill	Scenario	Target	Observed result	Assessment
find_object	ambiguous_cup	cup	ABORTED, failure code <code>ambiguous</code>	Pass
navigate_to	path_blocked	kitchen	ABORTED, failure code <code>path_blocked</code>	Pass
wave_greet	social_wave_unavailable	person	ABORTED, failure code <code>motion_unavailable</code>	Pass

Table 7.2: Preliminary fake-skill scenario results from the action-level runtime probe.

These results validate the fake-skill layer as a usable substrate for later planner-level testing. They do not yet prove that the planner always reacts correctly to those failures. The next

validation step is to run the same scenarios through planner-request injection and full user-turn injection, then check whether the planner asks for clarification, replans, or fails safely according to the expected policy.

7.4 Planner Request Admission

The same runtime pass exposed a planner-admission risk. One fixture request, `goal_probe`, was processed and produced an `ask_clarification` dialogue act, which was the expected response to ambiguous goal text. Later fixture requests were visible on `/planner/request`, but no corresponding planner request log was observed in the same observation window. This suggests that the goal-state helper or request-admission path may remain in a waiting state after the first clarification.

Probe	Observation	Interpretation
<code>goal_probe</code>	Planner received the request and emitted <code>ask_clarification</code> .	Expected behaviour for ambiguous goal text.
<code>goal_nav_1</code> and <code>goal_exec_1</code>	Requests were visible on <code>/planner/request</code> , but planner processing was not observed in the window.	Degraded result. Request admission or goal-state transition requires further testing.

Table 7.3: Planner request-admission observations from the preliminary runtime pass.

7.5 Speech Ingress and User-Turn Responsiveness

A speech-ingress probe published a tracked anonymous speaker and then a user utterance. The dialogue manager received the speech and issued a chatbot request. However, no corresponding `/chatbot_llm/turn_trace` event was observed during the window. This is a degraded result for full user-turn validation because it prevents a complete trace from speech input to chatbot response. It does not invalidate the planner-request or fake-skill action results, but it means that full end-to-end scoring should remain provisional until the user-turn trace is reliable.

7.6 Results Matrix

Table 7.4 summarises the current validation status against the methodology in Chapter 6.

Validation area	Status	Current evidence
Runtime launch profile	Pass	Planner, chatbot, orchestrator, trace viewer, and fake skill server were active in the live stack.
Fake-skill action scenarios	Pass	Ambiguous object search, blocked navigation, and unavailable wave scenarios returned expected failure codes.
Planner clarification on ambiguous goal	Partial pass	One ambiguous fixture produced <code>ask_clarification</code> .
Planner admission across repeated requests	Degraded	Later planner requests were visible on the topic but not processed in the observed window.
Full user-turn trace	Degraded	Speech reached <code>dialogue_manager</code> , but the expected chatbot turn trace was not observed.
Composite fake-skill planner suite	Not yet complete	The deterministic substrate exists, but repeated planner-level suite results are still pending.
Physical NAO execution comparison	Not yet complete	Current evidence focuses on controlled fake-skill and stack-level validation.

Table 7.4: Current validation status matrix.

7.7 Interpretation

The preliminary results support the architectural decision to separate fake execution, orchestrator feedback, and planner reasoning. The fake-skill layer can produce controlled failure payloads with scenario identifiers, which gives the planner a reproducible basis for recovery evaluation. The results also show why planner-only success is insufficient. A full HRI system must maintain reliable admission across multiple goals and preserve trace continuity from user speech to planner feedback and final dialogue.

The main unresolved issue is not the fake-skill substrate itself. The current risk lies in runtime coordination: request admission after clarification, speech-ingress trace completeness, and the remaining head-motion failure-injection seam. These findings define the next validation priorities before the final results chapter is closed.

Chapter 8

Discussion

8.1 Architectural Findings

The implementation indicates that a modular planner architecture is preferable to a direct dialogue-to-skill pipeline for this project. The planner does not own robot execution, speech output, or raw perception. It receives structured requests and returns structured plans or dialogue acts. This reduces the number of places where an LLM can accidentally bypass runtime safety constraints.

The most useful architectural seam is the boundary between `planner_llm` and `nao_orchestrator`. The planner can propose a sequence, but the orchestrator validates and dispatches it. This makes it possible to test the planner with fake skills, physical skills, or unavailable skills without changing the planner prompt itself. It also means that execution feedback can be represented as a first-class input to replanning rather than as an unstructured error message.

8.2 Planner Behaviour

The planner design is strongest when the task can be represented as a sequence of available skill contracts. Atomic commands, object search, navigation, and report-generation tasks map naturally to the registry. Composite instructions are harder because they require the planner to preserve order, carry evidence between steps, and decide whether a later step should still run after an earlier failure.

The preliminary validation evidence shows one expected clarification behaviour, but also suggests a request-admission risk after a clarification state. This is an important runtime finding. A planner-mediated robot must handle a sequence of user goals, not only the first accepted request. Future validation should therefore include repeated-goal tests, superseded-goal tests, and clarification-follow-up tests.

8.3 Grounding and World State

Grounding remains a central limitation. The architecture distinguishes `knowledge_snapshot`, `scene_summary`, and `state_t0`, which is useful because each object has a different role. The chatbot needs a compact prompt-level world state, the planner needs actionable entity identifiers and targets, and operators need a traceable scene summary. However, this separation only helps if the facts are fresh and stable at the time of the user turn.

The validation methodology therefore treats simulator or KnowledgeCore object insertion as a required probe. If a fact reaches KnowledgeCore but disappears before the next turn, the problem is a grounding freshness issue. If it reaches `grounded_context` but the answer ignores it, the problem is likely prompt or model behaviour. This distinction prevents perception noise from being misclassified as planner failure.

8.4 ROS4HRI Modularity

The stack is partly robot-agnostic and partly NAO-specific. The planner request, planner output, execution feedback, and dialogue act contracts are not inherently tied to NAO. The same architecture could be reused with another ROS-compatible platform if an equivalent skill registry and orchestrator adapter were provided. By contrast, speech execution, replay motion, posture control, head motion, and look-at behaviour depend on NAO-specific packages or adapters.

This separation is a practical strength. It allows the thesis to evaluate the planner loop independently from the physical robot while still preserving a credible path to robot execution. The fake-skill layer is especially valuable because it provides deterministic execution evidence while the physical skills remain subject to hardware, safety, and perception variability.

8.5 Validation Implications

The preliminary results show why the final evaluation should combine planner-request injection and full user-turn injection. Planner-request injection is reliable for testing plan validity and fake-skill recovery. Full user-turn injection is necessary for testing route correctness, dialogue ownership, and exactly-once speech. A thesis evaluation based on only one of these modes would either miss HRI integration faults or make planner scoring unnecessarily noisy.

Chapter 9

Limitations and Future Work

9.1 Current Limitations

The current system has several limitations. The physical implementation is centred on NAO, so cross-robot portability is architectural rather than fully proven. LLM planning remains sensitive to prompt design, model availability, token limits, and JSON schema adherence. The validation strategy is scenario-based and cannot prove exhaustive safety. Perception and grounding quality also constrain planner performance because a correct planner cannot reliably act on objects that are missing, stale, or ambiguously represented in the world state.

The current evidence base is also incomplete. The fake-skill substrate has been validated for selected deterministic action-level scenarios, but the full repeated planner-level suite still needs to be run. The user-turn path has shown degraded trace continuity in at least one live-stack pass. The planner request-admission path after clarification requires further focused testing.

9.2 Future Work

Future work should stabilise the planner-gate and goal-state transitions, then run the full fake-skill validation suite with repeated seeds and saved evidence bundles. The next implementation step is to make per-case metrics and aggregate reports a routine output of validation rather than a manual analysis product.

The architecture should also be extended in four directions: richer grounding through a persistent world model, broader skill-registry generation from reviewed contracts, safer composite and macro-skill execution, and cross-platform validation on another ROS-compatible robot or simulator. A longer-term extension would integrate a World Model Enricher that tracks uncertainty, object identity, and temporal freshness explicitly rather than passing only a compact snapshot to the planner.

Chapter 10

Conclusion

This thesis has presented the design and implementation of a modular LLM-based planner for a ROS4HRI-compatible NAO robot stack. The central architectural decision is to keep dialogue, grounding, planning, deterministic execution, and speech ownership separated by explicit runtime contracts. This separation allows the planner to use natural-language reasoning while the robot-facing stack continues to validate skill names, arguments, execution feedback, and dialogue acts through inspectable ROS interfaces.

The implementation demonstrates how a chatbot can route execution-looking turns to a planner, how the planner can produce structured plans over a skill registry, and how an orchestrator can act as the execution boundary between generated plans and robot or fake-skill adapters. The runtime contracts make the system more auditable than a direct prompt-to-action design because planner requests, grounded context, generated plans, execution feedback, and dialogue acts can be captured and checked independently.

The preliminary validation evidence supports the usefulness of the fake-skill substrate for controlled evaluation. Deterministic scenarios for ambiguous object search, blocked navigation, and unavailable social motion returned the expected structured failure codes. These results make it possible to evaluate recovery behaviour without depending on physical robot variability. The same validation pass also exposed integration risks in repeated planner request admission and full user-turn trace continuity. These are useful findings because they identify the remaining runtime seams that must be stabilised before the system can be assessed as a complete end-to-end HRI planner.

Overall, the work supports a measured conclusion: LLM planning can be integrated into ROS4HRI more safely when it is constrained by symbolic context, skill contracts, deterministic orchestration, and structured feedback. The approach does not remove the need for careful runtime validation, but it gives that validation clear seams and reproducible evidence.

Appendix A

Full Runtime Contracts

The full runtime contracts are described in Chapter 4. This appendix is reserved for additional machine-readable schemas when the final validation artefacts are frozen.

A.1 Runtime Skill Inventory

Table A.1 gives the compressed runtime projection used by the current planner and orchestrator profiles. The values are taken from the shared skill registry and are shortened only to fit the appendix. The machine-readable registry and the generated runtime projections are frozen with the source revision of each validation bundle. The inventory includes all sixteen planner-visible runtime skills, including the non-physical communication skill **ask_user**. Its inclusion does not move final wording or speech ownership out of the dialogue layer described by Contract 8.

Table A.1: Compressed runtime skill inventory and registry values.

Skill and category	Required parameters	Expected effect or evidence	Known failure modes	Executor and status
scan perception	none	refreshed scene evidence and typed scan payload	no_person, objects_only, backend_unavailable, unexpected_payload	orchestrator.scan; first-party
find_object perception	target	object or person evidence is returned	not_found, ambiguous, backend_unavailable	fake.find_object; fake
navigate_to navigation	target	robot arrives at the target location	path_blocked, unknown_location, safety_disabled	fake.navigate_to; fake
walk_to navigation	none	short local walking segment is executed or dry-run	invalid_local_goal, safety_disabled, path_blocked, naoqi_move_failed	orchestrator.walk_to; hybrid
bring_object manipulation	target, recipient	object is delivered to recipient or destination	acquisition_failure, navigation_failure, delivery_blocked, recipient_unavailable	fake.bring_object; fake
place_object manipulation	target, destination	object is released and a support relation is added	destination_unavailable, invalid_support, no_held_object, placement_failed	fake.place_object; fake
pick_object manipulation	target	robot holds the object and updates its support relation	object_unavailable, unreachable, already_held, grasp_failed	fake.pick_object; fake
perform_motion embodiment	object	robot posture or head orientation changes	unsupported_motion, dispatch_failed	orchestrator.perform_motion; first-party
look_at attention	none	robot gaze is redirected or reset	missing_target, dispatch_failed	orchestrator.look_at; first-party
kb_add knowledge	statements	requested predicates are added through KnowledgeCore	missing_statements, kb_service_unavailable, kb_mutation_rejected	orchestrator.kb_add; first-party
kb_revise knowledge	statements	requested predicates are revised through KnowledgeCore	missing_statements, kb_service_unavailable, kb_mutation_rejected	orchestrator.kb_revise; first-party
kb_remove knowledge	statements	requested predicates are removed through KnowledgeCore	missing_statements, kb_service_unavailable, kb_mutation_rejected	orchestrator.kb_remove; first-party
report_result communication	none	user receives a result summary through robot speech	missing_summary, say_dispatch_failed	orchestrator.report_result; first-party
ask_user communication	none	structured clarification request obtains missing user information	missing_prompt, dialogue_relay_unavailable	orchestrator.ask_user; first-party
wave_greet social motion	none	robot performs a greeting wave gesture	motion_unavailable, safety_disabled, unexpected_payload	orchestrator.wave_greet; hybrid
inspect_area perception	target	scene evidence for the target area is returned	ambiguous, area_unknown, backend_unavailable	fake.inspect_area; fake

The inventory separates the planner-facing name from the executor mapping. In the display column, `orchestrator.*` abbreviates `nao_orchestrator.*`, and `fake.*` abbreviates `fake_skills.*`. The table also records whether failure injection is supported in the current fake profile, which is used by the multi-step deterministic fake-skill cases in Chapter 6. The table does not imply that every hybrid or fake entry is enabled in the primary scored launch profile.

Appendix B

Launch Commands

```
1 ros2 launch nao_chatbot nao_chatbot_sim_demo.launch.py \  
2   start_planner_llm:=true \  
3   chatbot_planner_mode_enabled:=true \  
4   planner_llm_think:=false
```

Appendix C

Validation Log Template

Field	Value
Scenario ID	<i>S1–S8</i>
User command	<i>Paste command</i>
Model	<i>Model name</i>
Planner request	<i>Path or pasted payload</i>
Planner output	<i>Path or pasted payload</i>
Execution feed-back	<i>Path or pasted payload</i>
Dialogue act	<i>Path or pasted payload</i>
Outcome	<i>success / failure / recovered / clarification</i>
Notes	<i>Observations</i>

Table C.1: Validation log template.

Bibliography

- Ahn, Michael, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, et al. (2022). “Do As I Can, Not As I Say: Grounding Language in Robotic Affordances”. In: *arXiv preprint arXiv:2204.01691*.
- Bastianelli, Emanuele, Domenico D. Bloisi, Roberto Capobianco, Guglielmo Gemignani, Luca Iocchi, and Daniele Nardi (2013). “Knowledge representation for robots through human-robot interaction”. In: *arXiv preprint arXiv:1307.7351*.
- Brohan, Anthony, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, et al. (2023). “RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control”. In: *arXiv preprint arXiv:2307.15818*.
- Driess, Danny, Fei Xia, Mehdi S. M. Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, et al. (2023). “PaLM-E: An Embodied Multimodal Language Model”. In: *arXiv preprint arXiv:2303.03378*.
- Erol, Kutluhan, James Hendler, and Dana S. Nau (1994). “HTN Planning: Complexity and Expressivity”. In: *Proceedings of the Twelfth National Conference on Artificial Intelligence*. AAAI Press, pp. 1123–1128.
- Fong, Terrence, Illah Nourbakhsh, and Kerstin Dautenhahn (2003). “A survey of socially interactive robots”. In: *Robotics and Autonomous Systems* 42:3–4, pp. 143–166.
- Georgievski, Ilche and Marco Aiello (2014). “An Overview of Hierarchical Task Network Planning”. In: *arXiv preprint arXiv:1403.7426*.
- Goodrich, Michael A. and Alan C. Schultz (2007). “Human-robot interaction: a survey”. In: *Foundations and Trends in Human-Computer Interaction* 1.3, pp. 203–275.
- Gouaillier, David, Vincent Hugel, Pierre Blazevec, Chris Kilner, Jérôme Monceaux, Pascal Lafourcade, Brice Marnier, Julien Serre, and Bruno Maisonnier (2008). “The NAO humanoid: a combination of performance and affordability”. In: *arXiv preprint arXiv:0807.3223*.
- Grassi, Lucrezia, Carmine Tommaso Recchiuto, and Antonio Sgorbissa (2021). “Knowledge triggering, extraction and storage via human-robot verbal interaction”. In: *arXiv preprint arXiv:2104.11170*.
- Huang, Wenlong, Pieter Abbeel, Deepak Pathak, and Igor Mordatch (2022a). “Language Models as Zero-Shot Planners: Extracting Actionable Knowledge for Embodied Agents”. In: *International Conference on Machine Learning*.
- (2022b). “Language Models as Zero-Shot Planners: Extracting Actionable Knowledge for Embodied Agents”. In: *arXiv preprint arXiv:2201.07207*.

- Huang, Wenlong, Fei Xia, Ted Xiao, Harris Chan, Jacky Liang, Pete Florence, Andy Zeng, et al. (2022c). “Inner Monologue: Embodied Reasoning through Planning with Language Models”. In: *arXiv preprint arXiv:2207.05608*.
- Kim, Moo Jin, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. (2024). “OpenVLA: An Open-Source Vision-Language-Action Model”. In: *arXiv preprint arXiv:2406.09246*.
- Li, Junnan, Dongxu Li, Silvio Savarese, and Steven Hoi (2023). “BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models”. In: *arXiv preprint arXiv:2301.12597*.
- Liang, Jacky, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng (2022). “Code as Policies: Language Model Programs for Embodied Control”. In: *arXiv preprint arXiv:2209.07753*.
- Liu, Bo, Yuqian Jiang, Xiaohan Zhang, Qiang Liu, Shiqi Zhang, Joydeep Biswas, and Peter Stone (2023a). “LLM+P: Empowering Large Language Models with Optimal Planning Proficiency”. In: *arXiv preprint arXiv:2304.11477*.
- Liu, Haotian, Chunyuan Li, Qingyang Wu, and Yong Jae Lee (2023b). “Visual Instruction Tuning”. In: *arXiv preprint arXiv:2304.08485*.
- Macenski, Steve, Tully Foote, Brian Gerkey, Chris Lalancette, and William Woodall (2022). “Robot Operating System 2: Design, architecture, and uses in the wild”. In: *Science Robotics* 7.66, eabm6074.
- McDermott, Drew, Malik Ghallab, Adele Howe, Craig Knoblock, Ashwin Ram, Manuela Veloso, Daniel Weld, and David Wilkins (1998). *PDDL: The Planning Domain Definition Language*. Tech. rep. CVC TR-98-003/DCS TR-1165. Yale Center for Computational Vision and Control.
- Mohamed, Youssef and Séverin Lemaignan (2020). “ROS for Human-Robot Interaction”. In: *arXiv preprint arXiv:2012.13944*.
- Nau, Dana S., Tsz-Chiu Au, Okhtay Ilghami, Ugur Kuter, J. William Murdock, Dan Wu, and Fusun Yaman (2003). “SHOP2: An HTN Planning System”. In: *Journal of Artificial Intelligence Research* 20, pp. 379–404.
- Open Robotics (2025). *Robot Operating System 2 Documentation*. Accessed as project background documentation.
- Quigley, Morgan, Ken Conley, Brian Gerkey, Josh Faust, Tully Foote, Jeremy Leibs, Rob Wheeler, and Andrew Y. Ng (2009). “ROS: an open-source Robot Operating System”. In: *ICRA Workshop on Open Source Software*.
- ROS4HRI Community (2025). *ROS4HRI: ROS for Human-Robot Interaction*. Accessed as project background documentation.
- Saxena, Ashutosh, Ashesh Jain, Ozan Sener, Aditya Jami, Dipendra K. Misra, and Hema S. Koppula (2014). “RoboBrain: Large-scale knowledge engine for robots”. In: *arXiv preprint arXiv:1412.0691*.
- Shinn, Noah, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao (2023). “Reflexion: Language Agents with Verbal Reinforcement Learning”. In: *arXiv preprint arXiv:2303.11366*.

- Tenorth, Moritz and Michael Beetz (2013). “KnowRob: A knowledge processing infrastructure for cognition-enabled robots”. In: *The International Journal of Robotics Research* 32.5, pp. 566–590.
- (2017). “Representations for robot knowledge in the KnowRob framework”. In: *Artificial Intelligence* 247, pp. 151–169.
- Valmeekam, Karthik, Matthew Marquez, Sarath Sreedharan, and Subbarao Kambhampati (2023). “On the Planning Abilities of Large Language Models: A Critical Investigation”. In: *arXiv preprint arXiv:2305.15771*.
- Yao, Shunyu, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao (2022). “ReAct: Synergizing Reasoning and Acting in Language Models”. In: *arXiv preprint arXiv:2210.03629*.